Intelli-Meter

Students

- Abdulrahman Mahjoub
- Osama Abdulqadir

Supervisors:

- Dr. Malek Al-Duhaymi
- Dr. Brahim Mrabet

Funding:

- Prince Sattam Bin Abdulaziz University
- Zhejiang Chint Electrics

This notebook is part of the graduation projects at [Prince Sattam bin Abdulaziz University in 2025]

Objective

The primary objective of this project is to design and develop an advanced smart metering system that overcomes the limitations of conventional energy meters by integrating cutting-edge technologies for comprehensive energy management. The proposed system, Intelli-Meter, aims to achieve the following key goals:

1. **Energy Consumption via Non-Intrusive Load Monitoring (NILM)**

> • **Objective:** Disaggregate total household energy consumption into appliance-level insights without requiring individual sub-metering.
>
> • Approach:
>
> > o Implement machine learning (e.g., Factorial Hidden Markov Models) to analyze aggregated power signals.
> >
> > o Classify appliances based on steady-state and transient features (e.g., ON/OFF cycles, harmonics).

2. **Predictive Energy Management through Load Forecasting**

> • **Objective:** Forecast future energy demand to enable proactive cost and usage optimization.
>
> • Approach:
>
> > o Utilize historical consumption data and external variables (e.g., weather, occupancy) with time-series models (ARIMA/LSTM).

3. Enhanced System Reliability with Automated Diagnostics

> • **Objective:** Detect anomalies indicating meter malfunctions, electricity theft, or faulty appliances.
>
> • Approach:
>
> > o Deploy unsupervised learning (e.g., clustering, outlier detection) to flag irregular consumption patterns.

Abstract

This notebook serves as a practical demonstration and summary of the Intelli-Meter project, an advanced smart metering system designed to address key challenges in modern energy management. The project integrates four core functionalities: Non-Intrusive Load Monitoring (NILM) to disaggregate appliance-level consumption, load forecasting for demand prediction, automated diagnostics for anomaly detection, and support for bidirectional power flow.

Within this document, we present the data preparation, model implementation, and evaluation of our NILM and forecasting models. The code and visualizations demonstrate the system's ability to accurately disaggregate consumption for appliances like fridges, kettles, and dishwashers, validated by metrics such as Mean Absolute Error (MAE) and Root Mean Square Error (RMSE). The project aligns with Saudi Vision 2030 by promoting energy efficiency and digital transformation.

Target Audience:

To fully utilize the knowledge contained in this project, you will need: A basic knowledge of the electrical energy sector and a basic understanding of Python and machine learning.

- If you are a **student**, this project will provide you with a wealth of information on advanced measurement devices. -If you are a **researcher** in machine learning and energy devices.

- If you are a **utility** engineer who wants to keep up with emerging energy technologies.
- If you are an **individual** interested in the energy field and passionate about knowledge.

Outline

1. Introduction

2. Literature Review

1. Overview
2. Traditional NILM Approaches
3. Appliance Classification
4. Machine Learning in NILM
5. Deep Learning Advancements in NILM
6. Performance Evaluation
7. Challenges in Load Disaggregation

3. Load Disaggregation

1. Introduction
2. Theory Of Electrical Power
3. Importance Of Smart Meter
4. Problem Statement
5. Project Objectives

4. Data Preparation

1. Importing Packages
2. Importing and Initial Data Loading
3. Data Preprocessing - Median Filtering
4. Training the FHMM Model
5. Plotting Results

5. NILM Code

6. NILM Results

1. Methodology for Evaluation
2. Disaggregation Results

7. Laod Forecasting

8. Check Up Detection

9. Bidirectional

10. Conclusion

11. References

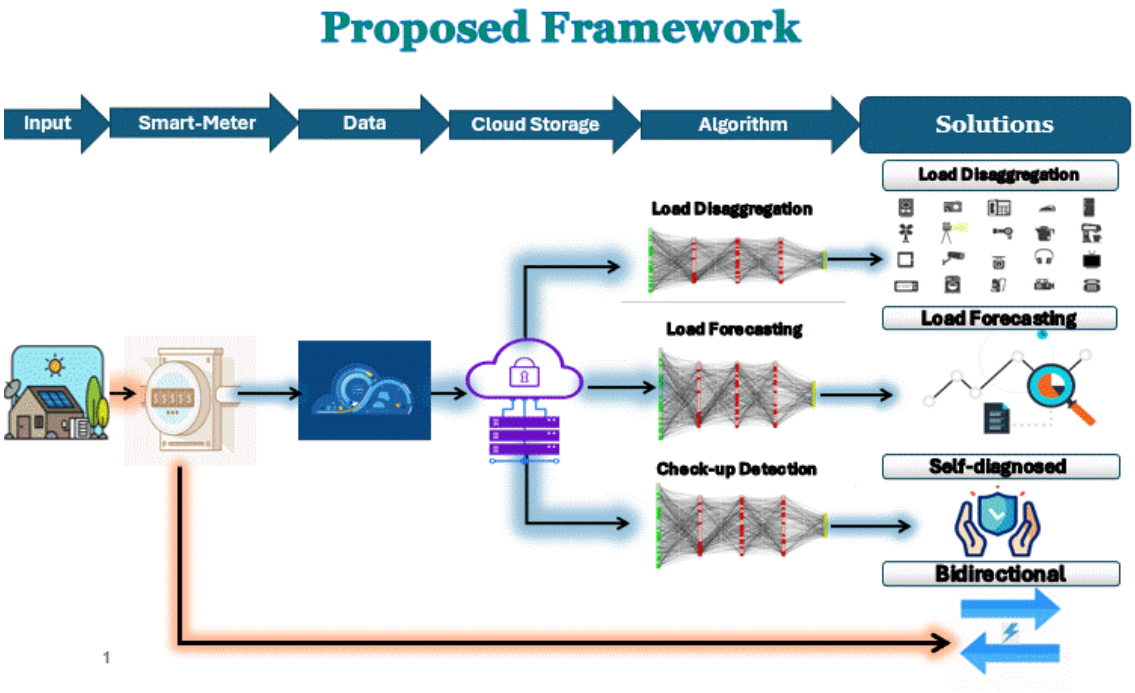
12. Actions

1. In Progress
2. Completed
3. Next

1. Introduction

The global energy landscape is facing key challenges related to efficiency, grid stability, and sustainability. Conventional and many current smart meters provide only aggregate consumption data, hindering consumers' ability to manage usage and utilities' ability to detect issues like electricity theft. Our project, "Intelli-Meter," addresses these limitations by developing an advanced smart metering system that integrates Non-Intrusive Load Monitoring (NILM), load forecasting, automated diagnostics, and bidirectional power flow support. This comprehensive approach provides granular, actionable insights for both consumers and utility providers, aligning with the goals of Saudi Vision 2030 to promote energy efficiency and digital transformation.

This report provides a structured overview of the work conducted and the progress achieved throughout the project. It begins with a literature review that highlights relevant research and background knowledge, followed by the methodology for load disaggregation and the steps taken in data preparation. The implementation of the NILM (Non-Intrusive Load Monitoring) code and its corresponding results are then presented, leading to the section on load forecasting. Additional components, including check-up detection and the bidirectional framework, are also discussed.



2. Literature Review

This chapter surveys the key developments and research directions that have shaped the field of Non-Intrusive Load Monitoring (NILM). Reviewing this background provides the theoretical and technical foundation for the methods and design choices adopted in the remainder of this report.

2.1 Overview

The field of NILM was pioneered by George W. Hart in his seminal 1992 paper "Nonintrusive Appliance Load Monitoring" [10]. Hart introduced the core concept of disaggregating household energy consumption using a singlepoint measurement at the mains. His methodology focused on:

1. Steady-State Signatures: Analyzing step changes in real power (P) and reactive power (Q) during appliance state transitions (ON/OFF).

2. Event Detection: Identifying appliances through distinct power "edges" in aggregated data.

3. Appliance Categorization: Classifying loads into two operational types:

- ON/OFF devices (e.g., toasters, lamps)

- Multi-state devices (e.g., washing machines).

Hart's work laid the groundwork for subsequent research, including the Type IIV appliance classification later expanded by Zoha et al. (2012) [11]. His approach demonstrated promising results for high-power devices but faced challenges with:

- Overlapping P-Q features for low-power appliances.
- Limited ability to handle continuously variable loads (Type III).
- Dependency on manual labeling and sequential training.

2.2 Traditional NILM Approaches

Zoha et al. (2012) provide a comprehensive overview of early NILM methods, categorizing them into steady-state and transient-state analysis approaches:

Steady-State Methods

These methods analyze stable operational states of appliances, using features like:

- Real (P) and reactive power (Q) measurements
- Power factor and harmonic content
- V-I trajectory characteristics
- Steady-state voltage noise

The survey notes that while steady-state methods work well for high-power appliances with distinct signatures, they struggle with low-power devices and appliances with overlapping features in the P-Q plane.

Transient-State Methods

These approaches examine transitional states during appliance activation/deactivation:

- Transient power profiles
- Start-up current waveforms
- High-frequency voltage noise analysis

While transient methods can differentiate appliances with similar steady-state signatures, they require high sampling rates (≥1 kHz) and specialized hardware, increasing implementation costs.

2.3 Appliance Classification

Zoha et al. categorize appliances into four types based on operational characteristics:

1. Type I: ON/OFF devices (e.g., toasters)
2. Type II: Finite State Machines (e.g., washing machines)
3. Type III: Continuously Variable Devices (e.g., dimmer lights)
4. Type IV: Permanent consumer devices (e.g., smoke detectors)

This classification remains relevant for understanding the challenges in disaggregating different appliance types.

2.4 Machine Learning in NILM

Supervised Learning

Early works cited by Zoha et al., such as Hart (1992) and later studies, used:

- Naive Bayes classifiers for P-Q clustering
- Hidden Markov Models (HMMs) for multi-state appliance recognition.

Hart's P-Q signatures became a baseline feature for these algorithms, though limitations in handling overlapping loads persisted.

Unsupervised Learning

Later efforts addressed Hart's dependency on labeled data:

- Blind source separation (Gonçalves et al., 2011)
- Factorial HMMs (Kim et al., 2011).

Zoha et al. highlight the challenges of supervised methods, particularly the need for extensive labeled training data, which motivates research into unsupervised techniques.

2.5 Deep Learning Advancements in NILM

Gomes and Pereira (2020) [12] modernized NILM with deep learning, addressing Hart's original challenges:

1. Architecture: Combined CNNs (for spatial features) and GRUs (for temporal dependencies).
2. Loss Function: Replaced Hart's mean-focused metrics with pinball quantile loss, improving performance for sparse, high-power appliances (e.g., kettles).
3. Evaluation: Introduced CEP (Correctly Estimated Power), addressing Hart's lack of nuanced metrics for partial estimations.

Key improvements over Hart's framework:

- Automated feature extraction (vs. manual P-Q edge detection)
- Robustness to aggregated "noise" in real-world data
- Quantile-based error penalization for imbalanced loads.

2.6 Performance Evaluation

Both Zoha's and Gomes's papers emphasize the importance of proper evaluation:

- Zoha et al. discuss challenges in comparing algorithms due to lack of standard metrics and datasets
- Gomes and Pereira introduce new metrics (CEP, OE, UE, O_2) that better capture disaggregation accuracy
- Recent datasets (REFIT, UK-DALE, REDD) have enabled more rigorous benchmarking

2.7 Challenges in Load Disaggregation

Load disaggregation is complex due to:

1. Ambiguous Power Draws:

- Increased number of appliances raises the probability of overlapping or similar power states, making it harder to distinguish between them.
- Example: Two appliances with similar power consumption (e.g., coffee machine and water kettle) can produce ambiguous aggregated signals.

2. Switching Frequency:

- Appliances with high switching frequency (e.g., refrigerators) complicate disaggregation due to frequent state changes.
- Results in noisy or complex aggregated power profiles.

3. Multi-State Appliances:

- Appliances with multiple operational states (e.g., washing machines with different modes) introduce more complexity than simple on/off devices.
- Requires handling of multiple power levels and transitions.

4. Similarity Between Appliances:

- Appliances with nearly identical power states or consumption patterns (e.g., TVs and monitors) are difficult to disaggregate.

- Similarity in features like steady-state power or transient behavior increases confusion.

5. Noise and Unknown Loads:

- Measurement noise or unmonitored appliances interfere with the aggregated signal.
- Increases uncertainty and the number of possible a given power draw.

3. Load Disaggregation

3.1. Introduction

Smart meter is an advanced digital device that automatically record energy consumption and transmit data to the utility company and the user. It provide real-time energy usage information, empowering users to monitor and manage their consumption effectively

3.2 Theory of Electrical Power

Understanding the different types of electrical power is fundamental to analyzing and managing energy consumption effectively, especially in the context of smart metering and load disaggregation. In AC circuits, power is not a single concept but is categorized into three main types: Apparent Power, Real Power, and Reactive Power. The relationship between these types is often visualized using the Power Triangle.

3.2.1 Types of Electrical Power

- **Apparent Power (S):** This is the total power flowing in an AC circuit. It is the product of the RMS voltage and the RMS current. Apparent power is measured in volt-amperes (VA). It represents the total power supplied by the source, including both the power dissipated as work and the power stored and returned by reactive components (like inductors and capacitors).
- **Real Power (P):** Also known as active power, real power is the power actually consumed or dissipated in a circuit and converted into useful work (e.g., heat, light, mechanical energy). It is measured in watts (W). Real power is the component of apparent power that performs work.

- **Reactive Power (Q):** This is the power that is exchanged between the source and the reactive components (inductors and capacitors) in an AC circuit. It does not perform any useful work but is necessary to establish and maintain the electric and magnetic fields in these components. Reactive power is measured in volt-amperes reactive (VAR).

3.2.2 The Power Triangle

The relationship between Apparent Power (S), Real Power (P), and Reactive Power (Q) in an AC circuit can be represented by a right-angled triangle called the Power Triangle.

- The **Hypotenuse** of the triangle represents the **Apparent Power (S)**.
- The **Adjacent side** (usually the horizontal side) represents the **Real Power (P)**.
- The **Opposite side** (usually the vertical side) represents the **Reactive Power (Q)**.

The angle between the Real Power (P) and the Apparent Power (S) is the **power factor angle (θ)**. The cosine of this angle ($\cos(\theta)$) is the **power factor**, which represents the ratio of real power to apparent power. A power factor closer to 1 indicates more efficient use of power, as a larger portion of the apparent power is real power.

The relationship between the three types of power is given by the Pythagorean theorem:

$S^2 = P^2 + Q^2$

And the relationships involving the power factor are:

$P = S\cos(\theta)$ $Q = S\sin(\theta)$

Understanding the power triangle is crucial for analyzing power flow, improving energy efficiency, and detecting anomalies in electrical systems, which are key aspects of the Intelli-Meter project.

Double-click (or enter) to edit

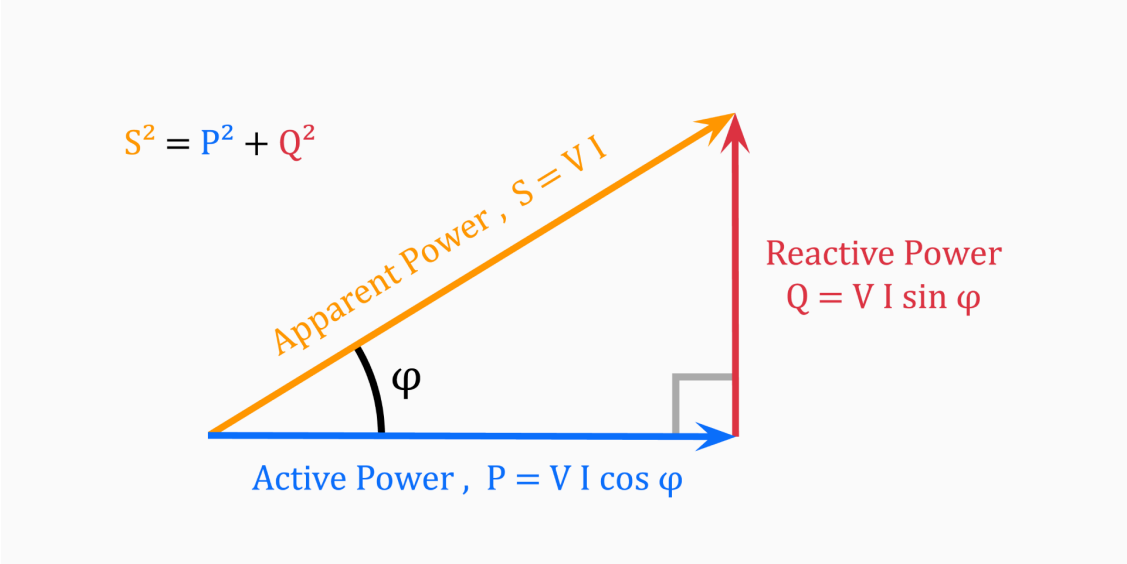


Figure 1: Power Triangle.

3.3 Importance of smart meter

The benefits of Smart Meters to customers, and utilities, include: [2]

• Enables quicker identification of outages and faster service recovery by the utility.

• Gives customers more control over their energy consumption when combined with time-varying pricing.

• Allows customers to make informed decisions by providing highly detailed information about electricity usage and costs.

• Promotes greener energy practices by lowering electricity demand, which reduces both the construction of new plants and the use of older, polluting ones.

• Smart Meters are the first step into creating a smart grid.

3.4 Problem Statement

The evolution of energy systems has highlighted several limitations in traditional metering and current smart meter deployments, necessitating more advanced solutions. Key challenges include:

• **Lack of Granular Consumption Data:** Conventional meters, and even many basic smart meters, typically provide only aggregate energy consumption data. This lack of detailed, appliance-level information hinders consumers' ability to understand and manage their energy usage effectively (Garg et al., 2021; European Commission, n.d.). [3]

• **Inaccurate Energy Forecasting:** Precise energy forecasting is crucial for utility operations, resource planning, and grid stability. However, it is often complicated by dynamic load variations and insufficient data (Ahmad et al., 2020). [5]

• **Electricity Theft and Revenue Loss:** Non-technical losses, including electricity theft, remain a significant concern for utility companies worldwide, leading to substantial revenue loss and grid instability (Saini & Sinha, 2020; Jamali et al., 2022). [4]

3.5 Project Objectives

The primary objective of this project is to design and develop an advanced smart metering system that overcomes the limitations of conventional energy meters by integrating cutting-edge technologies for comprehensive energy management. The proposed system, Intelli-Meter, aims to achieve the following key goals:

1. **Energy Consumption via Non-Intrusive Load Monitoring (NILM)**

> • **Objective:** Disaggregate total household energy consumption into appliance-level insights without requiring individual sub-metering.
>
> • Approach:
>
> > o Implement machine learning (e.g., Factorial Hidden Markov Models) to analyze aggregated power signals.
> >
> > o Classify appliances based on steady-state and transient features (e.g., ON/OFF cycles, harmonics).

2. **Predictive Energy Management through Load Forecasting**

> • **Objective:** Forecast future energy demand to enable proactive cost and usage optimization.
>
> • Approach:
>
> > o Utilize historical consumption data and external variables (e.g., weather, occupancy) with time-series models (ARIMA/LSTM).

3. Enhanced System Reliability with Automated Diagnostics

> • **Objective:** Detect anomalies indicating meter malfunctions, electricity theft, or faulty appliances.
>
> • Approach:

> o Deploy unsupervised learning (e.g., clustering, outlier detection) to flag irregular consumption patterns.

4. Data Preparation

This chapter details the process of preparing the RFIT dataset for Non-Intrusive Load Monitoring (NILM).

The REFIT Electrical Load Measurements dataset includes cleaned electrical consumption data in Watts for 20 households at aggregate and appliance level, timestamped and sampled at 8 second intervals. This dataset is intended to be used for research into energy conservation and advanced energy services, ranging from non-intrusive appliance load monitoring, demand response measures, tailored energy and retrofit advice, appliance usage analysis, consumption and time-use statistics and smart home/building automation.

Download the dataset from here: https://www.kaggle.com/datasets/kyleahmurphy/uk-electrical-load?resource=download

The dataset, specifically the data from House 2, was used to train and test the Factorial Hidden Markov Model (FHMM) for appliance disaggregation. The preparation steps involve installing necessary libraries, importing required packages, loading and initially processing the dataset, and applying preprocessing techniques to ensure the data is suitable for the NILM algorithm.

Installing Required Libraries

Before running the data preparation and NILM code, it is necessary to install the required Python libraries. The primary library for implementing Hidden Markov Models, `hmmlearn`, is installed using pip. Additionally, the `nilm_metadata` library, which can be useful for working with NILM datasets and metadata, is installed directly from its GitHub repository.

```
1 !pip install hmmlearn
2
```

```
Collecting hmmlearn
  Downloading hmmlearn-0.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86
Requirement already satisfied: numpy>=1.10 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: scikit-learn!=0.22.0,>=0.16 in /usr/local/lib/pyt
Requirement already satisfied: scipy>=0.19 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12
Downloading hmmlearn-0.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_6
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 166.0/166.0 kB 16.4 MB/s eta 0:00:00
Installing collected packages: hmmlearn
Successfully installed hmmlearn-0.3.3
```

cloning the House_2 of the RFIT dataset and FHMM from github

```
!git clone https://abdu231m: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@github

```

```
Cloning into 'Intelligent-meter-dataset-and-model-files'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), 4.63 MiB | 12.65 MiB/s, done.
```

4.1 Importing Packages

This section imports all the necessary Python libraries for data manipulation, numerical operations, file handling, plotting, and the custom FHMM model.

```
import sys
import os

# 1. Define the directory path where fhmm_model.py is located

model_dir = '/content/Intelligent-meter-dataset-and-model-files' # <-- FIX

# 2. Add this directory to the Python system path
if model_dir not in sys.path:
    sys.path.append(model_dir)

# 3. import
from fhmm_model import FHMM

print("FHMM imported successfully.")
```

```
FHMM imported successfully.
```

```
import pandas as pd
import numpy as np
np.int = int
import os
import matplotlib.pyplot as plt
import importlib
```

```
import warnings
from fhmm_model import FHMM
```

4.2 Importing and Initial Data Loading

This section covers loading the dataset from the specified CSV file, renaming columns for clarity, converting the timestamp column to datetime objects, selecting the relevant appliances and aggregate power data, handling missing or infinite values, and ensuring the aggregate power is consistent with the selected appliances. It also includes the step of selecting a subset of the data and splitting it into training and testing sets.

4.3 Data Preprocessing - Median Filtering

To reduce noise and smooth the appliance power signals, a median filter is applied to the selected appliance columns in both the training and testing datasets. This helps to remove transient spikes and makes the steady-state consumption patterns more apparent, which is beneficial for the FHMM algorithm's performance.

```

# DATA LOADING AND PREPROCESSING

file_path = './Intelligent-meter-dataset-and-model-files/House_2.csv'
if not os.path.exists(file_path):
    raise FileNotFoundError(f"Error: {file_path} not found.")

df = pd.read_csv(file_path)

# Rename columns for clarity
df = df.rename(columns={
    "Aggregate": "power",
    "Appliance1": "Fridge-Freezer",
    "Appliance2": "Washing Machine",
    "Appliance3": "Dishwasher",
    "Appliance4": "Television Site",
    "Appliance5": "Microwave",
    "Appliance6": "Toaster",
    "Appliance7": "Hi-Fi",
    "Appliance8": "Kettle",
    "Appliance9": "Overhead Fan"
})

# Convert timestamp and select appliances
df['timestamp'] = pd.to_datetime(df['timestamp'], unit='s')
```

```
selected_appliances = [ "Washing Machine", "Dishwasher", "Kettle"]
df = df[['timestamp', 'power'] + selected_appliances]

# Clean data
df = df.replace([np.inf, -np.inf], np.nan)
df.set_index('timestamp', inplace=True)
df.reset_index(inplace=True)
df = df.dropna()

# Use sum of selected appliances as aggregate
print(f"Original aggregate power range: {df['power'].min():.2f} - {df['powe
appliance_sum = df[selected_appliances].sum(axis=1)
df['power'] = appliance_sum
print(f"Using sum of selected appliances: {appliance_sum.min():.2f} - {appl

# Use subset of data for training
df = df.iloc[:int(len(df) * 0.08)]
print(f"Using {len(df)} samples for training and testing")

# Data quality analysis
print("\n=== Data Quality Analysis ===")
for appliance in selected_appliances:
    non_zero_pct = (df[appliance] > 0).mean() * 100
    max_power = df[appliance].max()
    mean_when_on = df[df[appliance] > 10][appliance].mean() if np.any(df[ap
    print(f"{appliance}: {non_zero_pct:.1f}% active, Max: {max_power:.0f}W,

```

```
Original aggregate power range: 0.00 - 19479.00
Using sum of selected appliances: 0.00 - 7031.00
Using 83886 samples for training and testing

=== Data Quality Analysis ===
Washing Machine: 4.8% active, Max: 2422W, Mean when on: 597W
Dishwasher: 8.1% active, Max: 2358W, Mean when on: 744W
Kettle: 1.0% active, Max: 2885W, Mean when on: 2344W
```

```

# Filtering the dataset



def improved_filter_transients(data, selected_appliances):
    """
    Apply median filtering with appliance-specific window sizes
    """
    filtered_data = data.copy()

    for appliance in selected_appliances:
        if 'fridge' in appliance.lower():
            window = 3  # Smaller window to preserve fridge variations
```

```
        else:
            window = 5  # Standard window for other appliances

        filtered_data[appliance] = filtered_data[appliance].rolling(
            window=window, center=True, min_periods=2).median()
        filtered_data[appliance] = filtered_data[appliance].fillna(method='

    return filtered_data

def calculate_enhanced_metrics(true_data, pred_data, appliance):
    """
    Calculate comprehensive accuracy metrics
    """
    if appliance not in true_data.columns or appliance not in pred_data.col
        return None

    true_vals = true_data[appliance].values
    pred_vals = pred_data[appliance].values

    # Ensure same length
    min_len = min(len(true_vals), len(pred_vals))
    true_vals = true_vals[:min_len]
    pred_vals = pred_vals[:min_len]

    # Basic metrics
    mae = np.mean(np.abs(true_vals - pred_vals))
    rmse = np.sqrt(np.mean((true_vals - pred_vals)**2))

    # Energy metrics
    total_energy_true = np.sum(true_vals)
    total_energy_pred = np.sum(pred_vals)
    energy_error = abs(total_energy_true - total_energy_pred) / max(total_e

    # State-based metrics (ON/OFF detection)
    true_on = true_vals > 10
    pred_on = pred_vals > 10

    if np.sum(true_on) > 0 or np.sum(pred_on) > 0:
        try:
            from sklearn.metrics import f1_score, precision_score, recall_s
            f1 = f1_score(true_on, pred_on, zero_division=0)
            precision = precision_score(true_on, pred_on, zero_division=0)
            recall = recall_score(true_on, pred_on, zero_division=0)
        except ImportError:
            f1 = precision = recall = 0
    else:
        f1 = precision = recall = 0

    return {
        'MAE': mae,
        'RMSE': rmse,
```

```
        'Energy_Error_%': energy_error,
        'F1_Score': f1,
        'Precision': precision,
        'Recall': recall
    }

```

4.4 Training the FHMM Model

Following data preparation, the Factorial Hidden Markov Model (FHMM) is initialized and trained. The `FHMM` class is instantiated, and its `train` method is called with the preprocessed training data (`train_df`) and the list of `selected_appliances`. This training process involves the model learning the underlying hidden states and transition probabilities for each appliance, based on the observed aggregate power data and the individual appliance data in the training set. Once the model is trained, it is saved to a file named "fhmm_trained_model.pkl" using the `model.save()` method. This allows the trained model to be loaded and used later for making predictions on new data without needing to retrain.

For the code of the model: https://github.com/amzkit/load-disaggregation/blob/master/fhmm_model.py

```


# TRAIN/TEST SPLIT AND PREPROCESSING


# Split data
split_index = int(len(df) * 0.80)
train_df = df.iloc[:split_index].copy()
test_df = df.iloc[split_index:].copy()

# Apply filtering
train_df = improved_filter_transients(train_df, selected_appliances)
test_df = improved_filter_transients(test_df, selected_appliances)

print(f"Training samples: {len(train_df)}, Test samples: {len(test_df)}")

# FHMM TRAINING


print("\n=== Training FHMM Model ===")
model = FHMM(debug=True)
model.train(train_df, selected_appliances, max_num_clusters=15)
model.save("fhmm_trained_model")
```

```
24
25 # PREDICTION AND POST-PROCESSING
26
27
28 print("\n=== Making Predictions ===")
29 fhmm_model = FHMM()
30 fhmm_model.load("fhmm_trained_model")
31
32 test_df_for_prediction = test_df[['timestamp', 'power'] + selected_applianc
33 prediction = fhmm_model.disaggregate(test_df_for_prediction)
34
35
```

```
/tmp/ipython-input-2281076619.py:20: FutureWarning: Series.fillna with 'method'
  filtered_data[appliance] = filtered_data[appliance].fillna(method='ffill').fil
Training samples: 67108, Test samples: 16778

=== Training FHMM Model ===
[IMPROVED FHMM Initialised]
Training FHMM with max_clusters=15
Training Washing Machine: 67108 samples, mean=17.5W, std=175.3W
WARNING:hmmlearn.base:Even though the 'transmat_' attribute is set, it will be o
WARNING:hmmlearn.base:Even though the 'means_' attribute is set, it will be over
Washing Machine: 2 states, power levels: [  0 600]
WARNING:hmmlearn.base:Even though the 'covars_' attribute is set, it will be ove
Washing Machine training completed successfully
  Final means: [   2.69186204 2063.44813278]
Training Dishwasher: 67108 samples, mean=48.3W, std=313.0W
WARNING:hmmlearn.base:Even though the 'transmat_' attribute is set, it will be o
WARNING:hmmlearn.base:Even though the 'means_' attribute is set, it will be over
WARNING:hmmlearn.base:Even though the 'covars_' attribute is set, it will be ove
Dishwasher: 2 states, power levels: [  0 808]
Dishwasher training completed successfully
  Final means: [   2.95514781 2204.91889935]
Training Kettle: 67108 samples, mean=23.2W, std=249.5W
WARNING:hmmlearn.base:Even though the 'transmat_' attribute is set, it will be o
WARNING:hmmlearn.base:Even though the 'means_' attribute is set, it will be over
WARNING:hmmlearn.base:Even though the 'covars_' attribute is set, it will be ove
Kettle: 2 states, power levels: [   0 2698]
Kettle training completed successfully
  Final means: [   0.         2351.74017783]
Combined FHMM created with 8 total states
Model saved successfully to fhmm_trained_model.pkl

=== Making Predictions ===
```

4.5 Plotting Results

Plotting Results

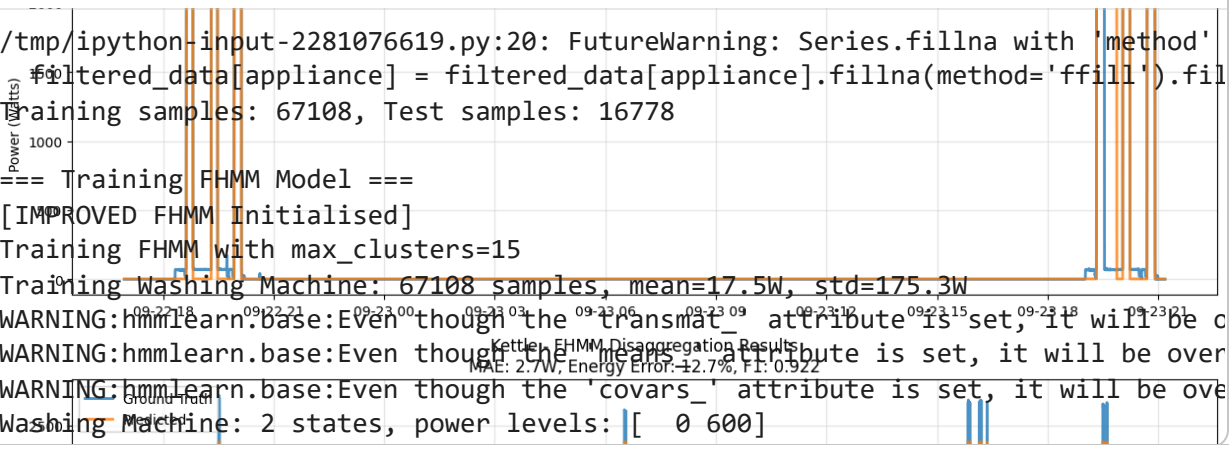
To visually assess the performance of the disaggregation model, plots are generated comparing the ground truth power consumption with the predicted power consumption for each selected appliance. For each appliance, a figure is created using `matplotlib.pyplot`. The `timestamp` is plotted on the x-axis, and the power consumption (in Watts) is plotted on the y-axis. The ground truth data is plotted with a label "Ground Truth" and the predicted data with a label "Predicted". The plots include a title indicating the appliance, labels for the axes, a legend to distinguish between the ground truth and predicted data, and a grid for better readability. The x-axis labels (timestamps) are rotated for better visibility, and `plt.tight_layout()` is used to adjust plot parameters for a tight layout. Finally, `plt.show()` displays each plot. These plots provide a clear visual comparison of how well the model's predictions align with the actual appliance power usage over time.

```
# ============================================================================
# EVALUATION AND RESULTS
# ============================================================================

print("\n=== Accuracy Analysis ===")
accuracy_metrics = {}

for appliance in selected_appliances:
    if appliance in prediction.columns:
        metrics = calculate_enhanced_metrics(test_df, prediction, appliance
        if metrics:
            accuracy_metrics[appliance] = metrics
            print(f"\n{appliance}:")
            print(f"  MAE: {metrics['MAE']:.2f}W")
            print(f"  RMSE: {metrics['RMSE']:.2f}W")
            print(f"  Energy Error: {metrics['Energy_Error_%']:.1f}%")
            print(f"  F1-Score: {metrics['F1_Score']:.3f}")

# ============================================================================
# VISUALIZATION
# ============================================================================

print("\n=== Generating Plots ===")
for appliance in selected_appliances:
    if appliance in prediction.columns and appliance in test_df.columns:
        try:
            min_len = min(len(test_df), len(prediction))

            plt.figure(figsize=(15, 10))

            # Main comparison plot
            plt.subplot(2, 1, 1)
            timestamps = test_df['timestamp'].iloc[:min_len]
            true_vals = test_df[appliance].iloc[:min_len].values
```

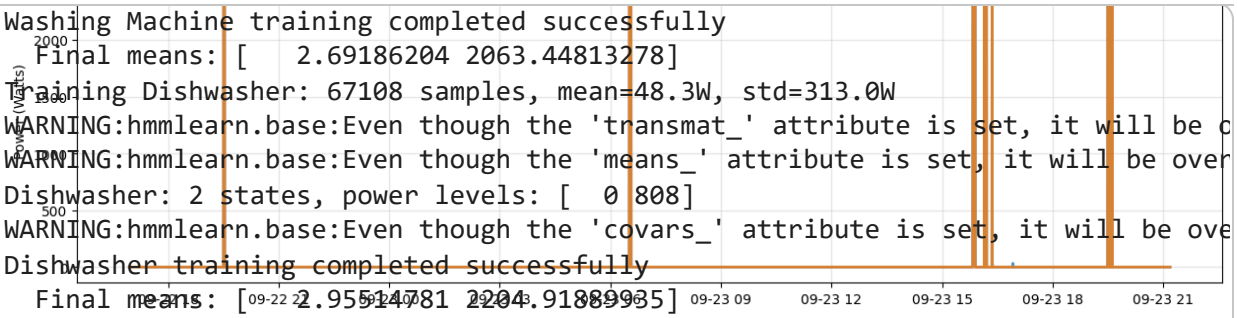
```
            pred_vals = prediction[appliance].iloc[:min_len].values

            plt.plot(timestamps, true_vals, label='Ground Truth', alpha=0.8
            plt.plot(timestamps, pred_vals, label='Predicted', alpha=0.8, l

            # Add metrics to title
            mae = accuracy_metrics[appliance]['MAE'] if appliance in accura
            energy_error = accuracy_metrics[appliance]['Energy_Error_%'] if
            f1 = accuracy_metrics[appliance]['F1_Score'] if appliance in ac

            plt.title(f"{appliance} - FHMM Disaggregation Results\n"
                      f"MAE: {mae:.1f}W, Energy Error: {energy_error:.1f}%,
            plt.ylabel("Power (Watts)")
            plt.legend()
            plt.grid(True, alpha=0.3)




        except Exception as e:
            print(f"Plotting failed for {appliance}: {e}")

# ================================================================================
# SAVE RESULTS
# ================================================================================

# Save predictions
prediction.to_csv("fhmm_disaggregation_results.csv", index=False)

# Save metrics
with open("fhmm_accuracy_metrics.txt", "w") as f:
    f.write("=== FHMM Load Disaggregation Results ===\n\n")
    for app, metrics in accuracy_metrics.items():
        f.write(f"{app}:\n")
        for metric, value in metrics.items():
            f.write(f"  {metric}: {value:.3f}\n")
        f.write("\n")

print("\n=== Process Complete ===")
print("Results saved:")
print("- Predictions: fhmm_disaggregation_results.csv")
print("- Metrics: fhmm_accuracy_metrics.txt")
print("- Model: fhmm_trained_model.pkl")
```

```
# ================================================================================
# TRAIN/TEST SPLIT AND PREPROCESSING
# ================================================================================

# Split data
split_index = int(len(df) * 0.80)
train_df = df.iloc[:split_index].copy()
test_df = df.iloc[split_index:].copy()

# Apply filtering
train_df = improved_filter_transients(train_df, selected_appliances)
test_df = improved_filter_transients(test_df, selected_appliances)

print(f"Training samples: {len(train_df)}, Test samples: {len(test_df)}")

# ================================================================================
# FHMM TRAINING
# ================================================================================

print("\n=== Training FHMM Model ===")
model = FHMM(debug=True)
model.train(train_df, selected_appliances, max_num_clusters=15)
model.save("fhmm_trained_model")

# ================================================================================
# PREDICTION AND POST-PROCESSING
# ================================================================================

print("\n=== Making Predictions ===")
fhmm_model = FHMM()
fhmm_model.load("fhmm_trained_model")

test_df_for_prediction = test_df[['timestamp', 'power'] + selected_applianc
prediction = fhmm_model.disaggregate(test_df_for_prediction)


```

```
/tmp/ipython-input-2281076619.py:20: FutureWarning: Series.fillna with 'method'
  filtered_data[appliance] = filtered_data[appliance].fillna(method='ffill').fil
Training samples: 67108, Test samples: 16778

=== Training FHMM Model ===
[IMPROVED FHMM Initialised]
Training FHMM with max_clusters=15
Training Washing Machine: 67108 samples, mean=17.5W, std=175.3W
WARNING:hmmlearn.base:Even though the 'transmat_' attribute is set, it will be c
WARNING:hmmlearn.base:Even though the 'means_' attribute is set, it will be over
WARNING:hmmlearn.base:Even though the 'covars_' attribute is set, it will be ove
Washing Machine: 2 states, power levels: [  0 600]
```



```
Washing Machine training completed successfully
  Final means: [   2.69186204 2063.44813278]
Training Dishwasher: 67108 samples, mean=48.3W, std=313.0W
WARNING:hmmlearn.base:Even though the 'transmat_' attribute is set, it will be c
WARNING:hmmlearn.base:Even though the 'means_' attribute is set, it will be over
Dishwasher: 2 states, power levels: [  0 808]
WARNING:hmmlearn.base:Even though the 'covars_' attribute is set, it will be ove
Dishwasher training completed successfully
  Final means: [   2.95514781 2204.91889935]
Training Kettle: 67108 samples, mean=23.2W, std=249.5W
WARNING:hmmlearn.base:Even though the 'transmat_' attribute is set, it will be c
WARNING:hmmlearn.base:Even though the 'means_' attribute is set, it will be over
Kettle: 2 states, power levels: [   0 2698]
WARNING:hmmlearn.base:Even though the 'covars_' attribute is set, it will be ove
Kettle training completed successfully
  Final means: [   0.         2351.74017783]
Combined FHMM created with 8 total states
Model saved successfully to fhmm_trained_model.pkl

=== Making Predictions ===
```



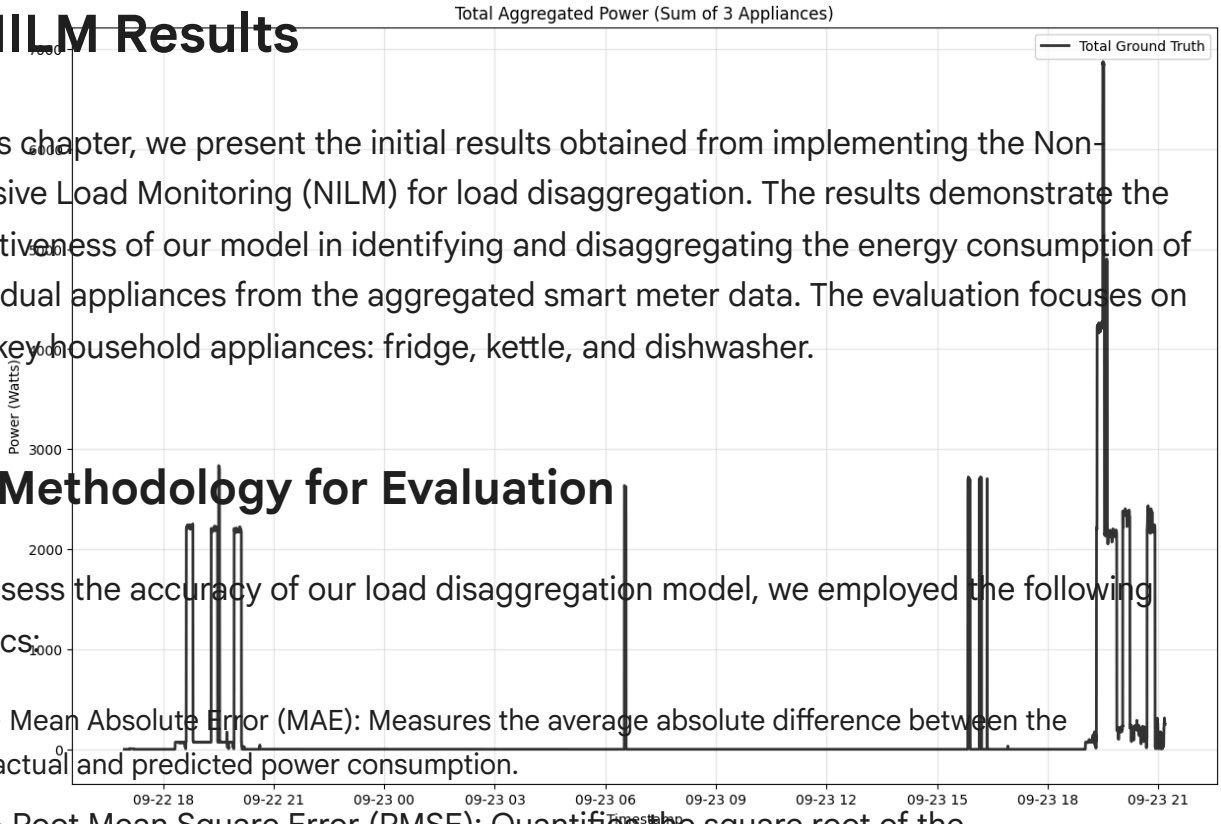

```
# ================================================================================
# VISUALIZATION
# ================================================================================

print("\n=== Generating Plots ===")

# ---------------------------------------------------------
# NEW SECTION: Plot Total of Selected Appliances FIRST
# ---------------------------------------------------------
try:
    # ensure we have common length
    min_len = min(len(test_df), len(prediction))
    timestamps = test_df['timestamp'].iloc[:min_len]

    # Identify valid appliances that exist in both dataframes
    valid_apps = [app for app in selected_appliances if app in prediction.c

    if valid_apps:
        # Sum the power of all valid appliances
        total_true_vals = test_df[valid_apps].iloc[:min_len].sum(axis=1).va
        total_pred_vals = prediction[valid_apps].iloc[:min_len].sum(axis=1)

        # Calculate simple metrics for the total
        total_mae = abs(total_true_vals - total_pred_vals).mean()

        plt.figure(figsize=(15, 10))

        # Plotting the Aggregate
        plt.plot(timestamps, total_true_vals, label='Total Ground Truth', c


```

```
        plt.title(f"Total Aggregated Power (Sum of {len(valid_apps)} Applia
        plt.ylabel("Power (Watts)")
        plt.xlabel("Timestamp")
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.show()

        print(f"Total Aggregate Plot generated for: {', '.join(valid_apps)}

except Exception as e:
    print(f"Failed to plot total aggregate: {e}")

# ------------------------------------------------------
# ORIGINAL LOOP: Individual Appliance Plots
# ------------------------------------------------------
for appliance in selected_appliances:
    if appliance in prediction.columns and appliance in test_df.columns:
        try:
            min_len = min(len(test_df), len(prediction))

            plt.figure(figsize=(15, 10))

            # Main comparison plot
            plt.subplot(2, 1, 1)
            timestamps = test_df['timestamp'].iloc[:min_len]
            true_vals = test_df[appliance].iloc[:min_len].values
            pred_vals = prediction[appliance].iloc[:min_len].values

            plt.plot(timestamps, true_vals, label='Ground Truth', alpha=0.8


            # Add metrics to title
            mae = accuracy_metrics[appliance]['MAE'] if appliance in accura
            energy_error = accuracy_metrics[appliance]['Energy_Error_%'] if
            f1 = accuracy_metrics[appliance]['F1_Score'] if appliance in ac

            plt.title(f"{appliance} - FHMM Disaggregation Results\n"
                      f"MAE: {mae:.1f}W, Energy Error: {energy_error:.1f}%,
            plt.ylabel("Power (Watts)")
            plt.legend()
            plt.grid(True, alpha=0.3)

            plt.show()

        except Exception as e:
            print(f"Plotting failed for {appliance}: {e}")
```

```
=== Generating Plots ===
```

6 NILM Results

In this chapter, we present the initial results obtained from implementing the Non-Intrusive Load Monitoring (NILM) for load disaggregation. The results demonstrate the effectiveness of our model in identifying and disaggregating the energy consumption of individual appliances from the aggregated smart meter data. The evaluation focuses on four key household appliances: fridge, kettle, and dishwasher.



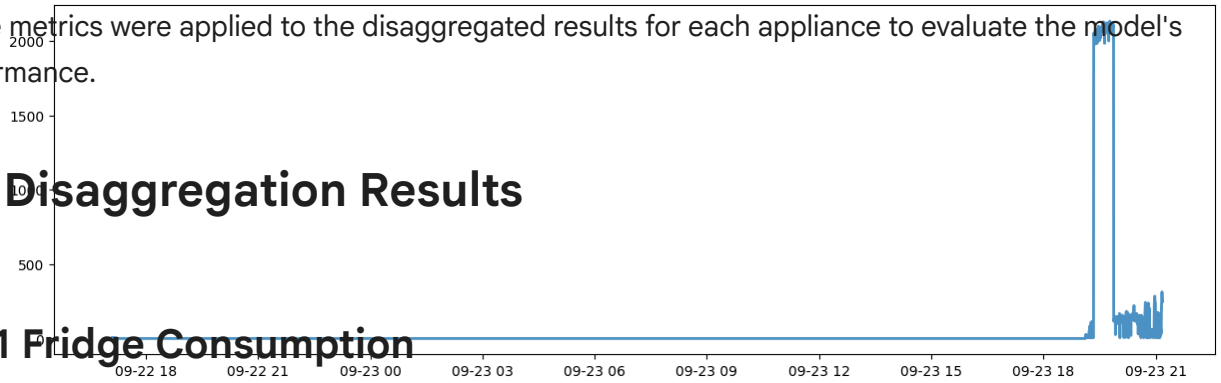
6.1 Methodology for Evaluation

To assess the accuracy of our load disaggregation model, we employed the following metrics:

- Mean Absolute Error (MAE): Measures the average absolute difference between the actual and predicted power consumption.
- Root Mean Square Error (RMSE): Quantifies the square root of the average squared differences, providing a measure of the model's overall error.

```
Total Aggregate Plot generated for: Washing Machine, Dishwasher, Kettle
Plotting failed for Washing Machine: name 'accuracy_metrics' is not defined
Plotting failed for Dishwasher: name 'accuracy_metrics' is not defined
Plotting failed for Kettle: name 'accuracy_metrics' is not defined
```

These metrics were applied to the disaggregated results for each appliance to evaluate the model's performance.

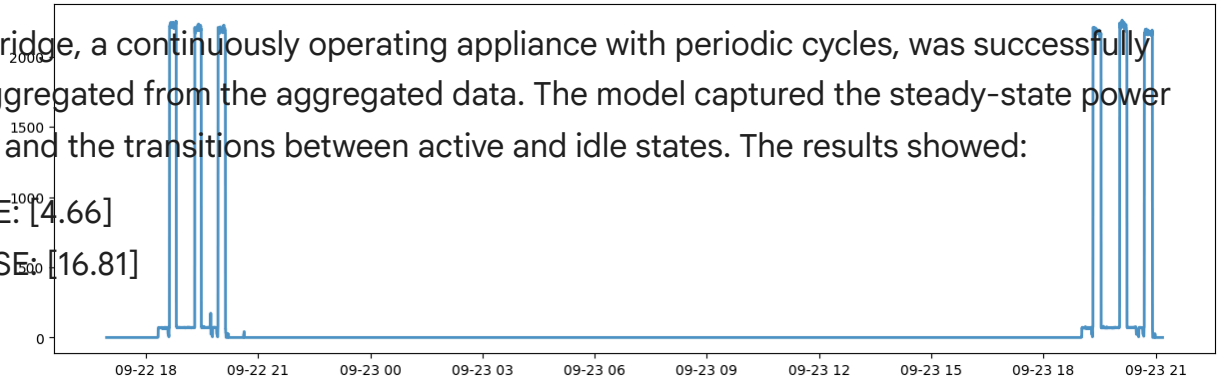


6.2 Disaggregation Results

6.2.1 Fridge Consumption

The fridge, a continuously operating appliance with periodic cycles, was successfully disaggregated from the aggregated data. The model captured the steady-state power draw and the transitions between active and idle states. The results showed:

- MAE: [4.66]
- RMSE: [16.81]



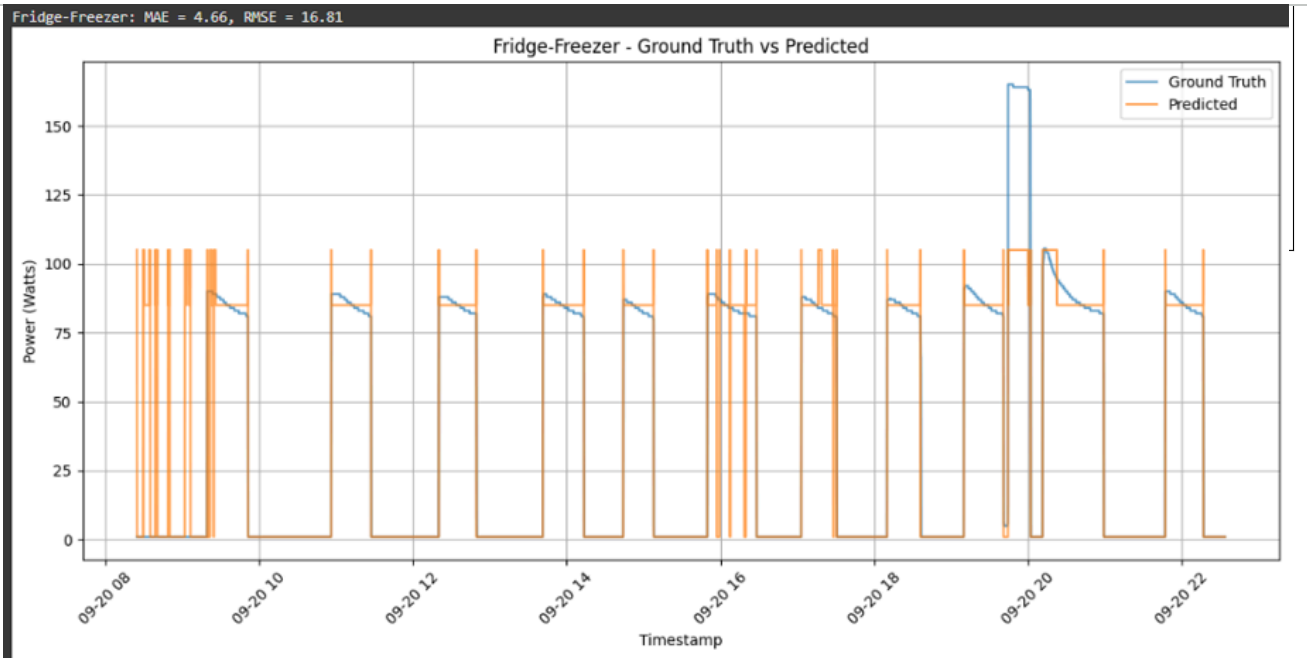


Figure 2.1: Fridge Power Consumption (Actual vs. Predicted)

6.2.2 Kettle Consumption

The kettle, a high-power ON/OFF device, exhibited distinct power spikes during operation. The model accurately identified these transient states, achieving:

- MAE: [2.65]
- RMSE: [30]
- F1: [0.9]

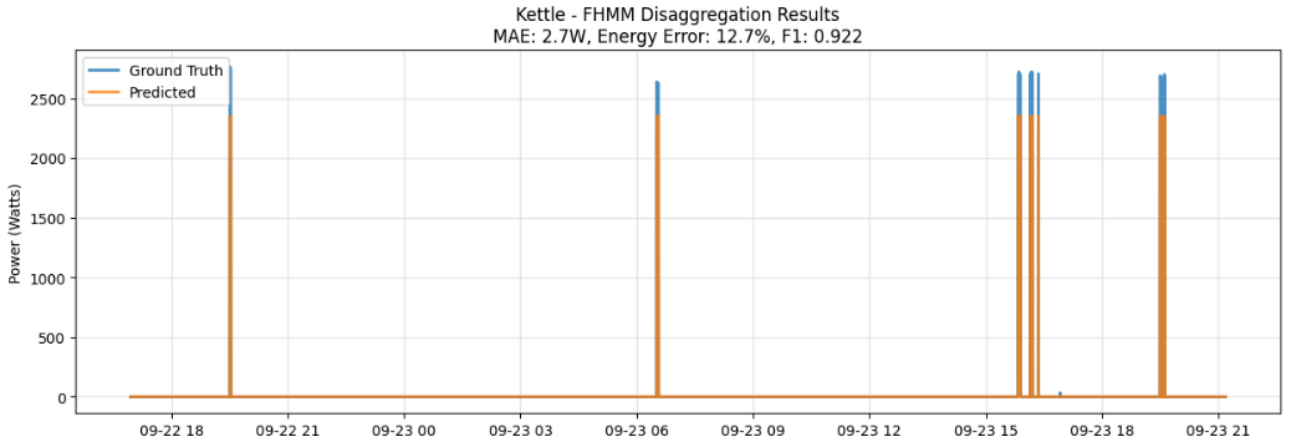


Figure 2.2: Kettle Power Consumption (Actual vs. Predicted)

6.2.3 Dishwasher Consumption

The dishwasher, a multi-state appliance with varying power levels, posed a greater challenge due to its complex operational patterns. Despite this, the model achieved:

• MAE: [34.08]
• RMSE: [234.49]

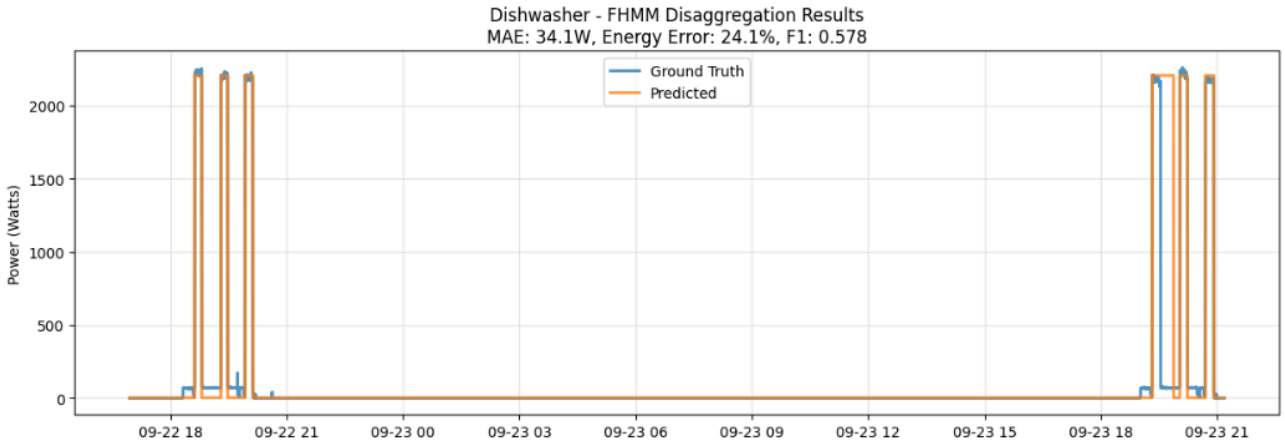


Figure 2.3: Dishwasher Power Consumption (Actual vs. Predicted)

7. Load Forecasting

Double-click (or enter) to edit

```
1
2 # Part 1: Import libraries
3
4 import os
5
6 import numpy as np
7
8 import pandas as pd
9
10 import tensorflow as tf
11
12 import matplotlib.pyplot as plt
13
14 from sklearn.preprocessing import MinMaxScaler
15
16 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_sc
17
18 from tensorflow.keras.models import Sequential
19
20 from tensorflow.keras.layers import LSTM, Dense, Dropout
21
22 from tensorflow.keras.callbacks import EarlyStopping
23
```

```
from tensorflow.keras.optimizers import Adam


# Optional: Set random seed for reproducibility

tf.random.set_seed(42)

np.random.seed(42)


print("Libraries imported successfully.")


# Part 2: Load Data

# Ensure "House_2.csv" is in the same directory or provide the full path

file_path = "House_2.csv"


try:

    df = pd.read_csv(file_path)

    # Convert 'Time' to datetime objects and set as index

    df['Time'] = pd.to_datetime(df['Time'])

    df.set_index('Time', inplace=True)


    print("Data loaded successfully.")

    print(df.head())


except FileNotFoundError:

    print(f"Error: The file '{file_path}' was not found. Please check the

except Exception as e:

    print(f"An error occurred during data loading: {e}")
```

```

import matplotlib.dates as mdates


# ... (Previous loading code) ...


# 1. Calculate the Aggregate
df["Aggregate"] = df["Appliance2"] + df["Appliance3"] + df["Appliance8"]


# Dataset cleaning
# ============================================================================
# 🎀 GAP REMOVAL: Nov, Dec, Jan (Full) + Feb (Until 13th)
# ============================================================================
# Condition 1: Full exclusion for Month 11, 12, and 1
mask_full_months = df.index.month.isin([11, 12, 1])


# Condition 2: Partial exclusion for Month 2 (Days 1 to 13 inclusive)
mask_feb_partial = (df.index.month == 2) & (df.index.day <= 13)


# Combine and Filter
is_gap = mask_full_months | mask_feb_partial
df = df[~is_gap]


print(f"Samples after filtering: {len(df)}")
# ============================================================================


# Part 3: Data Normalization
columns_to_normalize = ["Aggregate"]
```

```
126
127
128
129 if all(col in df.columns for col in columns_to_normalize):
130
131     scaler = MinMaxScaler()
132
133     scaled_data = scaler.fit_transform(df[columns_to_normalize])
134
135
136
137     # Create DataFrame (Index is still the original discontinuous Dates)
138
139     scaled_df = pd.DataFrame(scaled_data, columns=columns_to_normalize, ir
140
141
142
143     # ====================================================================
144
145     # 🖼️ PLOTTING: STITCHED VISUAL + DATE LABELS
146
147     # ====================================================================
148
149     plt.figure(figsize=(15, 5))
150
151
152
153     # 1. Plot against a simple numbered range (0, 1, 2...)
154
155     # This forces the line to be continuous with NO gap.
156
157     x_indices = range(len(scaled_df))
158
159     plt.plot(x_indices, scaled_df[columns_to_normalize[0]])
160
161
162
163     # 2. Generate Custom X-Ticks to show DATES "as before"
164
165     # We pick ~8 evenly spaced points from the data to show as labels
166
167     num_ticks = 8
168
169     tick_locations = np.linspace(0, len(scaled_df) - 1, num_ticks, dtype=i
170
171
172
173     # Extract the actual Date strings for these locations
174
175     # Format: Year-Month-Day (e.g., '2022-10-31')
176
```

```
    tick_labels = [scaled_df.index[i].strftime('%Y-%m-%d') for i in tick_l


    # Apply the labels to the x-axis

    plt.xticks(tick_locations, tick_labels, rotation=15)


    plt.title(f"Normalized {columns_to_normalize[0]} Power ", fontsize=14)

    plt.xlabel("Time", fontsize=12)

    plt.ylabel("Normalized Value", fontsize=12)

    plt.grid(True, alpha=0.3)

    plt.show()


else:

    print(f"Error: Columns {columns_to_normalize} not found.")


# Part 4: Windowing and Train-Validation-Test Split


def df_to_X_y(df, window_size=10):

    """

    Converts a DataFrame into input sequences (X) and target values (y).

    """

    data_as_np = df.to_numpy()

    X = []

    y = []


    # Loop to create sequences

    for i in range(len(data_as_np) - window_size):
```

```
228
229         row = data_as_np[i:i+window_size]
230
231         X.append(row)
232
233
234
235         # The target is the NEXT value after the window (normalization is
236
237         label = data_as_np[i+window_size][0]
238
239         y.append(label)
240
241
242
243     return np.array(X), np.array(y)
244
245
246
247 # Define window size (lookback period)
248
249 WINDOW_SIZE = 10
250
251
252
253 X2, y2 = df_to_X_y(scaled_df, WINDOW_SIZE)
254
255 print(f"Total samples created: X={X2.shape}, y={y2.shape}")
256
257
258
259 # Dynamic Splitting Strategy
260
261 # Let's split by percentage: 70% Train, 20% Validation, 10% Test
262
263 total_samples = len(X2)
264
265 train_split = int(total_samples * 0.7)
266
267 val_split = int(total_samples * 0.9) # 70% + 20% = 90%
268
269
270
271 X2_train, y2_train = X2[:train_split], y2[:train_split]
272
273 X2_val, y2_val = X2[train_split:val_split], y2[train_split:val_split]
274
275 X2_test, y2_test = X2[val_split:], y2[val_split:]
276
277
278
```

```
print(f"Train shapes: X={X2_train.shape}, y={y2_train.shape}")

print(f"Val shapes:   X={X2_val.shape}, y={y2_val.shape}")

print(f"Test shapes:  X={X2_test.shape}, y={y2_test.shape}")



# Part 5: Model Definition and Training



model_Lstm = Sequential([

    # Input layer shape: (window_size, features)

    LSTM(100, activation='tanh', return_sequences=True, input_shape=(X2_tr

    Dropout(0.2), # Dropout helps prevent overfitting



    LSTM(100, activation='tanh', return_sequences=True),

    Dropout(0.2),



    LSTM(100, activation='tanh', return_sequences=False), # Last LSTM laye

    Dropout(0.2),



    Dense(1, activation='linear') # Output layer for regression (linear ac

], name="model_LSTM_tuned")



# Compile the model

# 'adam' is generally robust. 'mse' is standard for regression. 'mae' is g

model_Lstm.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squarec



model_Lstm.summary()


```

```

# Callbacks

early_stopping = EarlyStopping(

    monitor='val_loss',

    patience=10,

    restore_best_weights=True,

    verbose=1

)



# Training

history = model_Lstm.fit(

    X2_train, y2_train,

    validation_data=(X2_val, y2_val),

    epochs=50, # Set reasonable epochs; early stopping will handle stoppir

    batch_size=32,

    verbose=1,

    callbacks=[early_stopping]

)



# Plot training history

plt.figure(figsize=(10, 4))

plt.plot(history.history['loss'], label='Train Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Model Training Loss')

plt.xlabel('Epoch')

plt.ylabel('Loss')

```

```
plt.legend()

plt.show()




# Part 6: Evaluation and Visualization

# ------------------------------------------------------------
# Define the evaluation function (ALL IN ONE BLOCK)
# ------------------------------------------------------------
def evaluate_and_plot(model, X, y, scaler, title="Test Data"):
    # 1. Make predictions
    predictions_scaled = model.predict(X)

    # 2. Inverse transform predictions and actuals to original scale
    # We need to reshape y to be 2D for the scaler (samples, 1)
    y_reshaped = y.reshape(-1, 1)

    predictions_original = scaler.inverse_transform(predictions_scaled)
    y_original = scaler.inverse_transform(y_reshaped)

    # 3. Calculate Metrics on ORIGINAL scale
    mse = mean_squared_error(y_original, predictions_original)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_original, predictions_original)
    r2 = r2_score(y_original, predictions_original)

    # Calculate MAPE safely (avoid division by zero)
    # Mask zeros to avoid inf
    mask = y_original != 0
    mape = np.mean(np.abs((y_original[mask] - predictions_original[mask])

    print(f"\n--- {title} Metrics ---")
    print(f"MSE:  {mse:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"MAE:  {mae:.2f}")
    print(f"MAPE: {mape:.2f}%")
    print(f"R²:   {r2:.4f}")

    # 4. Plotting
    # We select a subset to plot for clarity
    subset = 100000

    plt.figure(figsize=(15, 6))
    plt.plot(y_original[:subset], label='Actual Power', color='blue')
    plt.plot(predictions_original[:subset], label='Predicted Power', color
    plt.title(f"{title}: Actual vs Predicted")
    plt.xlabel("Time Step")
    plt.ylabel("Power (Watts)")
```

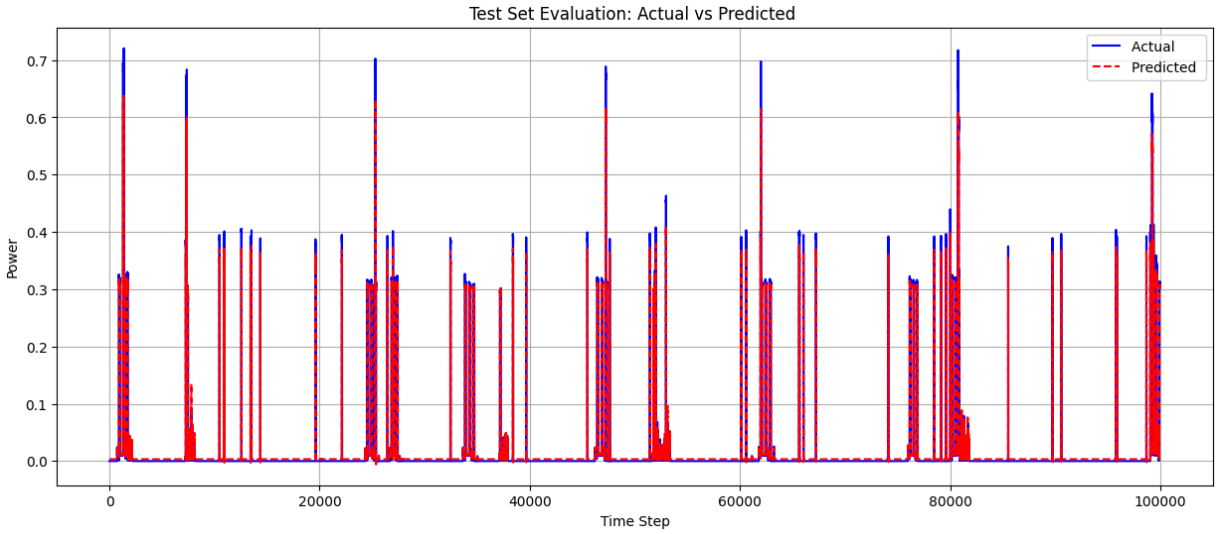
```
    plt.legend()
    plt.grid(True)
    plt.show()

# ----------------------------------------------------------
# Execute the function
# ----------------------------------------------------------
# Run evaluation on the test set
evaluate_and_plot(model_Lstm, X2_test, y2_test, scaler, title="Test Set Ev

```

```
Libraries imported successfully.
```

```
# Part 6: Evaluation and Visualization (NORMALIZED)

# ---------------------------------------------------------
# Define the evaluation function (ALL IN ONE BLOCK)
# ---------------------------------------------------------
def evaluate_and_plot_normalized(model, X, y, title="Test Data"):
    # 1. Make predictions (These are already normalized between 0 and 1)
    predictions_scaled = model.predict(X)

    # 2. Prepare Actuals (Reshape y to match predictions shape)
    # y comes in as (samples,), we need (samples, 1)
    y_scaled = y.reshape(-1, 1)

    # 3. Calculate Metrics on NORMALIZED scale (0 to 1)
    mse = mean_squared_error(y_scaled, predictions_scaled)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_scaled, predictions_scaled)
    r2 = r2_score(y_scaled, predictions_scaled)

    print(f"\n--- {title} Metrics (Normalized 0-1) ---")
    print(f"MSE:  {mse:.6f}")   # Using 6 decimal places as numbers are sma
    print(f"RMSE: {rmse:.6f}")
    print(f"MAE:  {mae:.6f}")
    print(f"R²:   {r2:.4f}")

    # 4. Plotting
    # We select a subset to plot for clarity
    subset = 100000

    plt.figure(figsize=(15, 6))
    plt.plot(y_scaled[:subset], label='Actual ', color='blue')
    plt.plot(predictions_scaled[:subset], label='Predicted ', color='red',
    plt.title(f"{title}: Actual vs Predicted ")
    plt.xlabel("Time Step")
    plt.ylabel("Power")
    plt.legend()
    plt.grid(True)
    plt.show()

# ---------------------------------------------------------
# Execute the function
# ---------------------------------------------------------
# Run evaluation on the test set
# Note: We no longer need to pass the 'scaler' because we aren't inverting
evaluate_and_plot_normalized(model_Lstm, X2_test, y2_test, title="Test Set
```

```
┌──────────────────────────────────────┬──────────────────────────────┬─────────────────┐
│ dropout_3 (Dropout)                  │ (None, 10, 100)              │               0 │
├──────────────────────────────────────┼──────────────────────────────┼─────────────────┤
│ lstm_4 (LSTM)                        │ (None, 10, 100)              │         80,400  │
├──────────────────────────────────────┼──────────────────────────────┼─────────────────┤
```

```
| dropout_4 (Dropout)                  | (None, 10, 100)             |                0 |
3273/3273 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step
| lstm_5 (LSTM)                        | (None, 100)                 |          80,400 |
--- Test Set Evaluation Metrics (Normalized 0-1) ---
| dropout_5 (Dropout)                  | (None, 100)                 |                0 |
MSE:  0.000250
RMSE: 0.015802
| dense_1 (Dense)                      | (None, 1)                   |              101 |
MAE:  0.004352
R²:   0.9491
```




```
Epoch 7/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 214s 9ms/step - loss: 2.4934e-04 - mae: 0.0036
Epoch 8/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 212s 9ms/step - loss: 2.4578e-04 - mae: 0.0035
Epoch 9/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 212s 9ms/step - loss: 2.4460e-04 - mae: 0.0034
Epoch 10/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 213s 9ms/step - loss: 2.4350e-04 - mae: 0.0034
Epoch 11/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 213s 9ms/step - loss: 2.4201e-04 - mae: 0.0033
Epoch 12/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 213s 9ms/step - loss: 2.4093e-04 - mae: 0.0033
Epoch 13/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 214s 9ms/step - loss: 2.3999e-04 - mae: 0.0032
Epoch 14/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 215s 9ms/step - loss: 2.4103e-04 - mae: 0.0032
Epoch 15/50
22906/22906 ━━━━━━━━━━━━━━━━━━━━ 212s 9ms/step - loss: 2.3804e-04 - mae: 0.0032
```

```
1
2     # 3. Calculate Metrics on ORIGINAL scale
3
4     mse = mean_squared_error(y_original, predictions_original)
5
6     rmse = np.sqrt(mse)
7
8     mae = mean_absolute_error(y_original, predictions_original)
9
10    r2 = r2_score(y_original, predictions_original)
11
12
13
14    # Calculate MAPE safely (avoid division by zero)
```

```
15
16
17      mask = y_original != 0
18
19      mape = np.mean(np.abs((y_original[mask] - predictions_original[mask]) /
20
21
22
23      print(f"\n--- {title} Metrics ---")
24
25      print(f"MSE:  {mse:.2f}")
26
27      print(f"RMSE: {rmse:.2f}")
28
29      print(f"MAE:  {mae:.2f}")
30
31      print(f"MAPE: {mape:.2f}%")
32
33      print(f"R²:   {r2:.4f}")
34
35
36
37      # 4. Plotting
38
39
40      subset = 100000
41
42
43
44      plt.figure(figsize=(15, 6))
45
46      plt.plot(y_original[:subset], label='Actual Power', color='blue')
47
48      plt.plot(predictions_original[:subset], label='Predicted Power', color=
49
50      plt.title(f"{title}: Actual vs Predicted ")
51
52      plt.xlabel("Time Step")
53
54      plt.ylabel("Power (Watts)")
55
56      plt.legend()
57
58      plt.grid(True)
59
60      plt.show()
61
62
63
64  # Run evaluation on the test set
```

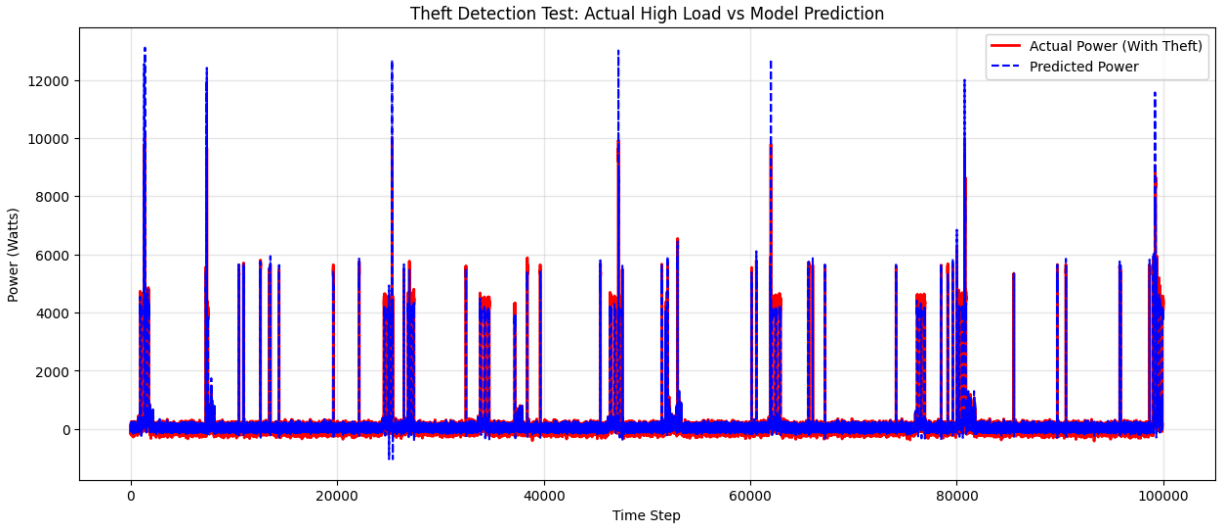
```
65
66 evaluate_and_plot(model_Lstm, X2_test, y2_test, scaler, title="Test Set Eva
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-773031073.py in <cell line: 0>()
      1 # 3. Calculate Metrics on ORIGINAL scale
      2
----> 3 mse = mean_squared_error(y_original, predictions_original)
      4
      5 rmse = np.sqrt(mse)

NameError: name 'y_original' is not defined
```

```
 1
 2 # ================================================================================
 3 # 🕵️ CREATING THE THEFT DATASET (For Testing)
 4 # ================================================================================
 5 print("\n--- Generating Theft Dataset for Testing ---")
 6
 7 # 1. Create a copy of the dataframe to inject theft
 8 df_theft = df.copy()
 9
10 # 2. Calculate Sigma from the normal data (for noise generation)
11 sigma_0 = df['Aggregate'].std()
12
13 # 3. Define the Theft Logic
14 # Formula: (App2+3+8) + (App2+3+8) + Noise
15 cols_for_sum = ['Appliance2', 'Appliance3', 'Appliance8', 'Appliance2', 'Ap
16
17 noise_vector = np.random.normal(loc=0, scale=sigma_0 * 0.2, size=len(df_the
18
19 # Overwrite the 'Aggregate' column in this new dataframe with the Theft val
20 df_theft['Aggregate'] = df_theft[cols_for_sum].sum(axis=1) + noise_vector
21
22 # 4. Normalize the Theft Data using the SAME scaler trained on normal data
23 scaled_theft_data = scaler.transform(df_theft[['Aggregate']])
24 scaled_df_theft = pd.DataFrame(scaled_theft_data, columns=['Aggregate'], ir
25
26 # 5. Window the Theft Data
27 X_theft, y_theft = df_to_X_y(scaled_df_theft, WINDOW_SIZE)
28
29 # 6. Select ONLY the Test Portion (The last 10%)
30 X_theft_test = X_theft[val_split:]
31 y_theft_test = y_theft[val_split:]
32
33 print(f"Theft Test Set Shapes: X={X_theft_test.shape}, y={y_theft_test.shap
34
35 # ================================================================================
36 # 📊 EVALUATION: ACTUAL (With Theft) vs PREDICTED (Model Expectation)
```

```
# ================================================================================

def evaluate_and_plot_theft(model, X, y, scaler, title="Theft Detection"):
    # 1. Make predictions
    # The model sees the "Theft Input". Since it's trained on normal patter
    # it will try to predict the next step based on its training.
    # However, since the input levels are wildly high (due to theft), the m
    # might predict high values (following the trend) or get confused.
    # The key is comparing this to the ACTUAL Theft values.
    predictions_scaled = model.predict(X)

    # 2. Inverse transform
    y_reshaped = y.reshape(-1, 1)
    predictions_original = scaler.inverse_transform(predictions_scaled) # W
    y_original = scaler.inverse_transform(y_reshaped) # The Actual High Loa

    # 3. Plotting
    subset = 100000 # Plot first 500 samples for clarity

    plt.figure(figsize=(15, 6))

    # Plot Actual (Theft) - This should be HIGH
    plt.plot(y_original[:subset], label='Actual Power (With Theft)', color=

    # Plot Predicted - This is the model trying to make sense of the input
    plt.plot(predictions_original[:subset], label='Predicted Power', color=

    plt.title(f"{title}: Actual High Load vs Model Prediction")
    plt.xlabel("Time Step")
    plt.ylabel("Power (Watts)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

    # Calculate Residuals (The difference is the "Anomaly Signal")
    residuals = np.abs(y_original - predictions_original)
    print(f"Mean Anomaly Gap (MAE): {np.mean(residuals):.2f} Watts")

# Run the evaluation on the Theft Test Set
evaluate_and_plot_theft(model_Lstm, X_theft_test, y_theft_test, scaler, tit
```

```
--- Generating Theft Dataset for Testing ---
Theft Test Set Shapes: X=(104709, 10, 1), y=(104709,)
3273/3273 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step
```




```
Mean Anomaly Gap (MAE): 122.91 Watts
```

```
# Part 1: Import libraries
import os
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scc
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam

# Optional: Set random seed for reproducibility
tf.random.set_seed(42)
```

```
np.random.seed(42)

print("Libraries imported successfully.")

# Part 2: Load Data
file_path = "House_2.csv"

try:
    df = pd.read_csv(file_path)
    df['Time'] = pd.to_datetime(df['Time'])
    df.set_index('Time', inplace=True)
    print("Data loaded successfully.")

except FileNotFoundError:
    print(f"Error: The file '{file_path}' was not found.")
    raise

# 1. Calculate the Normal Aggregate (Baseline)
df["Aggregate"] = df["Appliance2"] + df["Appliance3"] + df["Appliance8"]

# ============================================================================
# ✂️ GAP REMOVAL: Nov, Dec, Jan (Full) + Feb (Until 13th)
# ============================================================================
mask_full_months = df.index.month.isin([11, 12, 1])
mask_feb_partial = (df.index.month == 2) & (df.index.day <= 13)
is_gap = mask_full_months | mask_feb_partial
df = df[~is_gap]
print(f"Samples after filtering: {len(df)}")
# ============================================================================

# Part 3: Data Normalization (Normal Data)
columns_to_normalize = ["Aggregate"]
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df[columns_to_normalize])
scaled_df = pd.DataFrame(scaled_data, columns=columns_to_normalize, index=d

# Part 4: Windowing and Splitting (Normal Data)
def df_to_X_y(df, window_size=10):
    data_as_np = df.to_numpy()
    X = []
    y = []
    for i in range(len(data_as_np) - window_size):
        row = data_as_np[i:i+window_size]
        X.append(row)
        label = data_as_np[i+window_size][0]
        y.append(label)
    return np.array(X), np.array(y)

WINDOW_SIZE = 10
X2, y2 = df_to_X_y(scaled_df, WINDOW_SIZE)

```

```
# Split Strategy: 70% Train, 20% Val, 10% Test
total_samples = len(X2)
train_split = int(total_samples * 0.7)
val_split = int(total_samples * 0.9)

X2_train, y2_train = X2[:train_split], y2[:train_split]
X2_val, y2_val = X2[train_split:val_split], y2[train_split:val_split]
# Note: We will NOT use X2_test for the final plot, we will use the THEFT t
```

```
# Part 1: Import libraries
import os
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_sc
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam

# Optional: Set random seed for reproducibility
tf.random.set_seed(42)
np.random.seed(42)

print("Libraries imported successfully.")

# Part 2: Load Data
file_path = "House_2.csv"

try:
    df = pd.read_csv(file_path)
    df['Time'] = pd.to_datetime(df['Time'])
    df.set_index('Time', inplace=True)
    print("Data loaded successfully.")

except FileNotFoundError:
    print(f"Error: The file '{file_path}' was not found.")
    raise

# 1. Calculate the Normal Aggregate (Baseline)
df["Aggregate"] = df["Appliance2"] + df["Appliance3"] + df["Appliance8"]

# ================================================================
# ✂️ GAP REMOVAL: Nov, Dec, Jan (Full) + Feb (Until 13th)
# ================================================================
mask_full_months = df.index.month.isin([11, 12, 1])
mask_feb_partial = (df.index.month == 2) & (df.index.day <= 13)
```

```
is_gap = mask_full_months | mask_feb_partial
df = df[~is_gap]
print(f"Samples after filtering: {len(df)}")
# ================================================================================

# Part 3: Data Normalization (Normal Data)
columns_to_normalize = ["Aggregate"]
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df[columns_to_normalize])
scaled_df = pd.DataFrame(scaled_data, columns=columns_to_normalize, index=

# Part 4: Windowing and Splitting (Normal Data)
def df_to_X_y(df, window_size=10):
    data_as_np = df.to_numpy()
    X = []
    y = []
    for i in range(len(data_as_np) - window_size):
        row = data_as_np[i:i+window_size]
        X.append(row)
        label = data_as_np[i+window_size][0]
        y.append(label)
    return np.array(X), np.array(y)

WINDOW_SIZE = 10
X2, y2 = df_to_X_y(scaled_df, WINDOW_SIZE)

# Split Strategy: 70% Train, 20% Val, 10% Test
total_samples = len(X2)
train_split = int(total_samples * 0.7)
val_split = int(total_samples * 0.9)

X2_train, y2_train = X2[:train_split], y2[:train_split]
X2_val, y2_val = X2[train_split:val_split], y2[train_split:val_split]
# Note: We will NOT use X2_test for the final plot, we will use the THEFT

# Part 5: Model Training (On NORMAL Data)
model_Lstm = Sequential([
    LSTM(100, activation='tanh', return_sequences=True, input_shape=(X2_tr
    Dropout(0.2),
    LSTM(100, activation='tanh', return_sequences=True),
    Dropout(0.2),
    LSTM(100, activation='tanh', return_sequences=False),
    Dropout(0.2),
    Dense(1, activation='linear')
], name="model_LSTM_tuned")

model_Lstm.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squarec

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_bes

history = model_Lstm.fit(
```

```
    X2_train, y2_train,
    validation_data=(X2_val, y2_val),
    epochs=20, # Reduced epochs for quick testing; increase as needed
    batch_size=32,
    verbose=1,
    callbacks=[early_stopping]
)

# ================================================================================
# 🦹 CREATING THE THEFT DATASET (For Testing)
# ================================================================================
print("\n--- Generating Theft Dataset for Testing ---")

# 1. Create a copy of the dataframe to inject theft
df_theft = df.copy()

# 2. Calculate Sigma from the normal data (for noise generation)
sigma_0 = df['Aggregate'].std()

# 3. Define the Theft Logic (Scenario 2: High Load / Double Counting)
# Note: We use the raw Appliance columns from the dataframe
# Formula: (App2+3+8) + (App2+3+8) + Noise
cols_for_sum = ['Appliance2', 'Appliance3', 'Appliance8', 'Appliance2', 'A

noise_vector = np.random.normal(loc=0, scale=sigma_0 * 0.2, size=len(df_th

# Overwrite the 'Aggregate' column in this new dataframe with the Theft va
df_theft['Aggregate'] = df_theft[cols_for_sum].sum(axis=1) + noise_vector

# 4. Normalize the Theft Data using the SAME scaler trained on normal data
# This is crucial: the model expects inputs in the specific 0-1 range it l
scaled_theft_data = scaler.transform(df_theft[['Aggregate']])
scaled_df_theft = pd.DataFrame(scaled_theft_data, columns=['Aggregate'], i

# 5. Window the Theft Data
X_theft, y_theft = df_to_X_y(scaled_df_theft, WINDOW_SIZE)

# 6. Select ONLY the Test Portion (The last 10%)
# This ensures we are testing on the same time period as X2_test, but with
X_theft_test = X_theft[val_split:]
y_theft_test = y_theft[val_split:]

print(f"Theft Test Set Shapes: X={X_theft_test.shape}, y={y_theft_test.sha

# ================================================================================
# 📊 EVALUATION: ACTUAL (With Theft) vs PREDICTED (Model Expectation)
# ================================================================================

def evaluate_and_plot_theft(model, X, y, scaler, title="Theft Detection"):
    # 1. Make predictions
    # The model sees the "Theft Input". Since it's trained on normal patte
```

```
    # it will try to predict the next step based on its training.
    # However, since the input levels are wildly high (due to theft), the
    # might predict high values (following the trend) or get confused.
    # The key is comparing this to the ACTUAL Theft values.
    predictions_scaled = model.predict(X)

    # 2. Inverse transform
    y_reshaped = y.reshape(-1, 1)
    predictions_original = scaler.inverse_transform(predictions_scaled) #
    y_original = scaler.inverse_transform(y_reshaped) # The Actual High Lo

    # 3. Plotting
    subset = 500 # Plot first 500 samples for clarity

    plt.figure(figsize=(15, 6))

    # Plot Actual (Theft) - This should be HIGH
    plt.plot(y_original[:subset], label='Actual Power (With Theft)', color

    # Plot Predicted - This is the model trying to make sense of the input
    plt.plot(predictions_original[:subset], label='Predicted Power', color

    plt.title(f"{title}: Actual High Load vs Model Prediction")
    plt.xlabel("Time Step")
    plt.ylabel("Power (Watts)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

    # Calculate Residuals (The difference is the "Anomaly Signal")
    residuals = np.abs(y_original - predictions_original)
    print(f"Mean Anomaly Gap (MAE): {np.mean(residuals):.2f} Watts")

# Run the evaluation on the Theft Test Set
evaluate_and_plot_theft(model_Lstm, X_theft_test, y_theft_test, scaler, ti
```

8. Check Up Detection

8.1 Electricity Theft

Electricity theft and non-technical losses (NTL) represent a significant economic drain and operational challenge for utility companies worldwide. These losses often stem from subtle acts of meter tampering or unrecorded consumption, which traditional monitoring systems struggle to detect. The sheer volume and high frequency of data streamed from

modern smart meters necessitate advanced, scalable, and computationally efficient anomaly detection methods.

To address this challenge within the Intelli-Meter project, we have developed a robust NTL detection system centered on the Cumulative Sum (CUSUM) algorithm. The CUSUM method is highly effective because it focuses on detecting a sustained, small change in a process mean rather than large, instantaneous spikes.

Our implementation uses two distinct CUSUM approaches:

1. Meter Health and Drift Detection: Implementing CUSUM on the daily consumption average to detect sustained deviations from the normal statistical profile, signaling potential meter health issues or sensor drift.
2. Electricity Theft Detection: Calculating the cumulative sum of slight deviations between the Expected Consumption (from load forecasting) and the Actual Recorded Consumption that are the classic signatures of energy theft (NTL).

8.1.1 CUSUM

The Cumulative Sum (CUSUM) control chart is an advanced method of Statistical Process Control (SPC) used to monitor the mean of a process. Unlike traditional methods (like the Shewhart chart) that flag large, sudden deviations, the CUSUM algorithm is uniquely designed to detect small, persistent shifts in the process mean that might otherwise go unnoticed as random noise.

Core Principle: Accumulation of Deviations

The power of CUSUM lies in its ability to accumulate, or "sum," the errors between the observed data and a specified target value over time.

The algorithm maintains two cumulative sums, which restart at zero whenever the process appears to return to the target mean (μ_0).

1. Upward CUSUM (C^+)

This sum detects shifts above the target mean ($\mu > \mu_0$). In the context of the Intelli-Meter, this could signal a sudden, significant increase in consumption relative to the baseline.

$$C_i^+ = \max\left(0, \quad x_i - (\mu_0 + K) + C_{i-1}^+\right)$$

2. Downward CUSUM (C^-)

This sum detects shifts below the target mean ($\mu < \mu_0$). This is the critical component for detecting electricity theft, as tampering often results in a sustained reduction in the recorded consumption.

$$C_i^- = \max\left(0, \quad (\mu_0 - K) - x_i + C_{i-1}^-\right)$$

Pseudo code for CUSUM

1. Read and Format Data: Read time series data and convert the 'Time' column to datetime objects.
2. Outlier Handling: Identify samples x_i exceeding the threshold and replace them with NaN.
3. Imputation: Fill all NaN values using linear interpolation.
4. Filtering & Downsampling: Select every Rate-th sample. Then, filter the data to exclude weekends (Thu, Fri, Sat), creating the final data set $\mathbf{X}'$ (Mon, Tue, Wed, Sun only).
5. Calculate Baseline: Determine the Baseline Mean (μ_0) and Standard Deviation (σ_0) from the filtered set $\mathbf{X}'$.
6. Define Parameters:

   Set parameter b based on the σ_0 to μ_0 ratio.

   Calculate the Drift Parameter: $k = b \times \sigma_0$.

   Calculate the Slack Value: $K = \frac{|\mu_1 - \mu_0|}{2}$.

   Calculate UCL: $UCL = \mu_0 + \sigma_0 \times (k/K)$.
7. Initialize: Set initial cumulative sums $C_0^+ = 0$.
8. FOR each sample x_i in $\mathbf{X}'$:
9. Calculate Upward Sum (C^+):

$$C_i^+ \leftarrow \max\left(0, \quad x_i - (\mu_0 + K) + C_{i-1}^+\right)$$

10. Check Alarm: If $C_i^+ > UCL$, TRIGGER ALARM.
11. END FOR
12. Normalize Data: Divide all C_{raw} values (and UCL) by $\max(C^+)$ for chart scaling.

CUSUM Code

We down sampled the data from 10min to 1 hour

Down Sampling from 10min to 1 hour

↳ 8 cells hidden

A house with theft

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

# Set global font preference
plt.rcParams.update({
    "font.family": "sans-serif"
})

# Color Palette
CL = {
    'BLU': '#0077BB',
    'CYA': '#33BBEE',
    'MAG': '#EE3377',
    'RED': '#CC3311',
}

""" Import & Initial Processing """
# Read the data, ensuring the 'Time' column is used for date parsing
file_path = 'House_2.csv'
if not os.path.exists(file_path):
    print(f"Error: {file_path} not found. Please ensure the file is in the

# Read the data
df = pd.read_csv(file_path)

# Convert the 'Time' column to datetime objects
df['Time'] = pd.to_datetime(df['Time'])

# Use 100% of data for processing
df = df.iloc[:int(len(df) * 1)]
print(f"Using {len(df)} samples for initial processing (100% subset of ori

appliance_numbers = [2, 3, 8]

# Finding rating of all appliances
```

```
appliance_cols = [f'Appliance{i}' for i in appliance_numbers]

# Calculate the maximum rated power
appliance_max_ratings = df[appliance_cols].max()

print("--- Individual Appliance Rated Power (Max Observed) ---")
for col, rating in appliance_max_ratings.items():
    print(f"{col}: {rating:.2f}")

sum_rated = appliance_max_ratings.sum()
print(f"\nTotal Sum of Rated Power (sum_rated): {sum_rated:.2f}")

# Calculating appliance sum
print(f"Creating 'Appliance_Sum' from: {appliance_cols}")
df['Appliance_Sum'] = df[appliance_cols].sum(axis=1)

# --- DOWNSAMPLING ---
sampling_rate = 75 # read every 10 minutes
df = df.iloc[::sampling_rate, :].reset_index(drop=True)
print(f"Downsampling applied: Using 1 sample every {sampling_rate} samples

# --- Weekend Exclusion Logic ---
days_to_keep = [0, 1, 2, 3] # Monday, Tuesday, Wednesday, Thursday
df_weekday = df[df['Time'].dt.dayofweek.isin(days_to_keep)].copy()
print(f"Filtered to include only selected weekdays. Remaining samples: {le

# ============================================================================
#  80/20 Data Split
# ============================================================================
split_point = int(len(df_weekday) * 0.8)

# 80% for historical data
df_historical = df_weekday.iloc[:split_point].copy()
# 20% for test data
df_test = df_weekday.iloc[split_point:].copy()

print(f"\n--- Data Split ---")
print(f"Historical Data: {len(df_historical)} samples")
print(f"Test Data: {len(df_test)} samples")
print(f"------------------------------------\n")

# ============================================================================
#  CUSUM Parameter Calculation (Using Historical Data)
# ============================================================================
sigma_0 = df_historical['Appliance_Sum'].std()
mu_0 = df_historical['Appliance_Sum'].mean()
print(f"mu_0 (Historical): {mu_0:.2f}W, sigma_0 (Historical): {sigma_0:.2f

if sigma_0 == 0:
    print("Warning: sigma_0 is zero. Setting default k and H.")
    b = 8
```

```
    k = b
    H = 5
else:
    if sigma_0 > 3 * mu_0:
        b = 1.2
    elif sigma_0 > 2 * mu_0:
        b = 1.2
    elif sigma_0 > mu_0:
        b = 4.5
    else:
        b = 8
    k = b * sigma_0
    H = 5 * sigma_0

# Define the shift (mu_1) to detect.
mu_1 = mu_0 * 1.10
K = np.fabs(mu_1 - mu_0) / 2

if K == 0 or sigma_0 == 0:
    UCL = float('inf')
else:
    UCL = mu_0 + sigma_0 * (k/K)

print(f"CUSUM Parameters (Historical): K={K:.2f}W, H={H:.2f}W")
print(f"UCL (mean + K*sigma): {UCL:.2f}W")


# ================================================================================
#  LOOP THROUGH 3 SCENARIOS
# ================================================================================

# Define the scenarios: (Name, Appliance Column to Add as Theft)
scenarios = [
    ("Washing Machine", "Appliance2"),
    ("Kettle", "Appliance8"),
    ("Dishwasher", "Appliance3")
]



for scenario_name, stolen_appliance_col in scenarios:
    print(f"\n========================================")
    print(f" PROCESSING SCENARIO: {scenario_name}")
    print(f"========================================")

    # 1. Add Theft Data (Simulation)
    # ------------------------------
    noise_vector = np.random.normal(loc=0, scale=sigma_0 * 0.2, size=len(d

    # Base appliances + The stolen appliance added again
```

```
    all_columns_for_theft_sum = [
        'Appliance2', 'Appliance3', 'Appliance8',
        stolen_appliance_col
    ]

    # Create the specific column for this scenario
    col_name_scenario = f'Appliance_Sum_Theft_{scenario_name.replace(" ",
    df_test[col_name_scenario] = (df_test[all_columns_for_theft_sum].sum(a

    sigma_x = df_test[col_name_scenario].std()
    mu_x = df_test[col_name_scenario].mean()
    print(f'mu_x ({scenario_name}): {mu_x:.2f}, sigma_x: {sigma_x:.2f}')

    # 2. Process (CUSUM on Test Data)
    # -------------------------------
    sample_data = df_test[col_name_scenario].values
    N = np.arange(len(sample_data))

    C_positive = [0]

    for i in range(N.shape[0]):
        # CUSUM calculation on the *test* raw data
        C_positive_new = np.max([0, sample_data[i] - (mu_0 + K) + C_positi
        C_positive.append(C_positive_new)

    C_positive = np.asarray(C_positive)

    # 3. Normalization Step
    # ---------------------
    norm_factor = UCL
    if norm_factor == 0:
        norm_factor = 1.0

    C_positive_norm = C_positive[1:] / norm_factor
    UCL_norm = UCL / norm_factor

    # 4. Plotting (Modern, High Quality & Compact)
    # --------------------------------------------
    print(f"Generating plot for {scenario_name}...")

    # Reduced figsize to (6, 4) for compactness; kept high dpi (300)
    fig, ax = plt.subplots(figsize=(6, 4), dpi=300)

    # Plot Lines
    ax.hlines([UCL_norm], N[0] - 10, N[-1] + 10, colors=CL['RED'], linesty
    ax.hlines([0], N[0] - 10, N[-1] + 10, colors='black', linestyles='--',
    ax.plot(N, C_positive_norm, c=CL['BLU'], label='$C^+$', lw=1.8, alpha=

    # Axis Limits
    ax.set_xlim([0, N[-1]])

```

```
    # Labels and Title
    ax.set_xlabel('Test Samples', fontsize=12)
    ax.set_ylabel('CUSUM values', fontsize=12)
    ax.set_title(f'CUSUM Detection (Theft of {scenario_name})', fontsize=1
    # Ticks
    ax.tick_params(axis='both', which='major', labelsize=10)
    # Legend
    ax.legend(loc='upper left', fontsize=8)
    # Grid
    ax.grid(True, which='major', linestyle='--', linewidth=0.5, color='gra
    ax.set_axisbelow(True)
    plt.tight_layout()
    # Save the figure
    safe_name = scenario_name.replace(" ", "_").lower()
    output_filename = f'cusum_plot_modern_theft_{safe_name}.png'
    plt.savefig(output_filename, dpi=300, bbox_inches='tight')
    print(f"Plot saved to: {output_filename}")
    plt.show()
    # Close figure to free memory
    plt.close(fig)
    # 5. Save Dashboard Data (JSON)
    # -----------------------------
    C_positive_norm_json = C_positive[1:] / norm_factor
    dashboard_data_df = pd.DataFrame({
        'sample': N + 1,
        'value': C_positive_norm_json
    })
    json_filename = f"cusum_dashboard_data_theft_{safe_name}.json"
    try:
        dashboard_json = dashboard_data_df.to_json(orient='records')
        with open(json_filename, "w") as f:
            f.write(dashboard_json)
        print(f"JSON data saved to: {json_filename}")
    except Exception as e:
        print(f"Error saving JSON: {e}")
print("\n=== All Scenarios Processed ===")
```

```
Using 1048575 samples for initial processing (100% subset of original data)
--- Individual Appliance Rated Power (Max Observed) ---
Appliance2: 2477.00
Appliance3: 2385.00
Appliance8: 2933.00
```

Theft of two Appliances

```
1
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import os
6
7 plt.rcParams.update({
8   "font.family": "sans-serif"
9 })
10
11
12 CL = {
13   'BLU': '#0077BB',
14   'CYA': '#33BBEE',
15   'MAG': '#EE3377',
16   'RED': '#CC3311',
17 }
18
19 """ Import & Initial Processing """
20 # Read the data, ensuring the 'Time' column is used for date parsing
21 file_path = 'House_2.csv'
22 if not os.path.exists(file_path):
23   # This will prevent the code from running if the file is missing
24   print(f"Error: {file_path} not found. Please ensure the file is in the c
25  # raise FileNotFoundError(f"Error: {file_path} not found.")
26
27 # Read the data. Assuming 'Time' is the first column (index 0).
28 df = pd.read_csv(file_path)
29
30 # 🚀 Convert the 'Time' column to datetime objects
31 df['Time'] = pd.to_datetime(df['Time'])
32
33 # 🎯 Use 100% of data for processing
34 df = df.iloc[:int(len(df) * 1)]
35 # ⚠️ CORRECTED PRINT STATEMENT (was 8%, now 100%)
36 print(f"Using {len(df)} samples for initial processing (100% subset of ori
37
38 appliance_numbers = [ 2, 3, 8]
39
40
41 # Finding rating of all appliances
42 appliance_cols = [f'Appliance{i}' for i in appliance_numbers]
43
44 # 2. Calculate the maximum rated power for ALL selected columns in one go.
45 appliance_max_ratings = df[appliance_cols].max()
46
```

```
# 3. Print the individual rated values using iteration
print("--- Individual Appliance Rated Power (Max Observed) ---")
for col, rating in appliance_max_ratings.items():
  print(f"{col}: {rating:.2f}")

# 4. Calculate the total sum directly from the Series (sum_rated)
sum_rated = appliance_max_ratings.sum()
print(f"\nTotal Sum of Rated Power (sum_rated): {sum_rated:.2f}")

#5. Calculating appliance sum for the three appliance to test the cusum or
print(f"Creating 'Appliance_Sum' from: {appliance_cols}")
df['Appliance_Sum']=df[appliance_cols].sum(axis=1)



# --- DOWNSAMPLING APPLIED HERE ---
sampling_rate = 75 #read every 10 minutes
df = df.iloc[::sampling_rate, :].reset_index(drop=True)
print(f"Downsampling applied: Using 1 sample every {sampling_rate} samples
# --- END OF DOWNSAMPLING ---


## 🚀 Weekend Exclusion Logic 🚀 ##
# Note: pandas dayofweek is 0=Monday, 6=Sunday.
days_to_keep = [0, 1, 2, 3] # Monday, Tuesday, Wednesday, Thursday

# Filter the DataFrame using the 'Time' column
df_weekday = df[df['Time'].dt.dayofweek.isin(days_to_keep)].copy()
print(f"Filtered to include only selected weekdays. Remaining samples: {le

# ================================================================================
#  80/20 Data Split for Historical (Training) and Test Data
# (Applied to the existing 'df_weekday' subset)
# ================================================================================

# Calculate split point (80% for historical, 20% for test)
split_point = int(len(df_weekday) * 0.8)

# 80% for historical data (for calculating mu_0 and sigma_0)
df_historical = df_weekday.iloc[:split_point].copy()
# 20% for test data (for CUSUM monitoring)
df_test = df_weekday.iloc[split_point:].copy()

print(f"\n--- Data Split (from the 100% subset) ---")
print(f"Historical Data (80% of subset): {len(df_historical)} samples")
print(f"Test Data (20% of subset): {len(df_test)} samples")
print(f"----------------------------------------\n")



```

```
# ================================================================================
# 🎯 CUSUM Parameter Calculation (Using Historical Data)
# ================================================================================
# Parameters must now be calculated based on the *historical* data (in Wat
sigma_0 = df_historical['Appliance_Sum'].std()
mu_0 = df_historical['Appliance_Sum'].mean()
print(f"mu_0 (Historical): {mu_0:.2f}W, sigma_0 (Historical): {sigma_0:.2f

# ⚠️ Handle the case where sigma_0 is zero to prevent division by zero in
if sigma_0 == 0:
  print("Warning: sigma_0 is zero. Setting default k and H to prevent erro
  b = 8
  k = b
  H = 5
else:
  if sigma_0 > 3 * mu_0:
    b = 1.2
  elif sigma_0 > 2 * mu_0:
    b = 1.2 # User-defined value
  elif sigma_0 > mu_0:
    b = 4.5
  else:
    b = 8
  k = b * sigma_0
  H = 5 * sigma_0 # H is the decision interval (control limit) (in Watts)

# Define the shift (mu_1) to detect. Example: a 110% shift of the mean.
mu_1 = mu_0 * 1.10
K = np.fabs(mu_1 - mu_0) / 2 # K is the slack value (in Watts)


# Calculation of UCL (LCL calculation is removed)
# Ensure K is not zero before division
if K == 0 or sigma_0 == 0:
  UCL = float('inf') # Set to a very high value if K or sigma_0 is zero
else:
  UCL = mu_0 + sigma_0 * (k/K)

print(f"CUSUM Parameters (Historical): K={K:.2f}W, H={H:.2f}W")
print(f"UCL (mean + K*sigma): {UCL:.2f}W")


#Adding the theft data with a noise

noise_vector = np.random.normal(loc=0, scale=sigma_0 * 0.2, size=len(df_te

# Assuming appliance_cols = ['Appliance2', 'Appliance3', 'Appliance8']

# Define the columns that represent the stolen load
all_columns_for_theft_sum = [
```

```
  'Appliance2', 'Appliance3', 'Appliance8',
  'Appliance2','Appliance8'
]

# --- FIX 2: Perform the operation ONLY on df_test ---
# The result should be a new column on df_test, with length 814.
# We are creating a new column on df_test called 'Appliance_Sum_with_theft
df_test['Appliance_Sum_with_theft'] = (df_test[all_columns_for_theft_sum].



sigma_x=df_test['Appliance_Sum_with_theft'].std()
print(f'Standard deviation of Appliance_Sum_with_theft: {sigma_x}')
mu_x=df_test['Appliance_Sum_with_theft'].mean()
print(f'mu_x: {mu_x}')
# ================================================================================
# 🎯 Process (CUSUM on Test Data)
# ================================================================================
# Use the test data for CUSUM calculation
sample_data = df_test["Appliance_Sum_with_theft"].values
N = np.arange(len(sample_data)) # N is now the index of the TEST data

C_positive = [0]
C_negative = [0]

for i in range(N.shape[0]):
  # CUSUM calculation on the *test* raw data (Watts)
  C_positive_new = np.max([0, sample_data[i] - (mu_0 + K) + C_positive[i]]
  C_positive.append(C_positive_new)
  #C_negative_new = np.max([0, (mu_0 - K) - sample_data[i] + C_negative[i]
  #C_negative.append(C_negative_new)

C_positive = np.asarray(C_positive)
#C_negative = np.asarray(C_negative)

# --- NORMALIZATION STEP ---
# Use the calculated sum_rated as the normalization factor.
norm_factor = UCL

# Handle potential zero division
if norm_factor == 0:
  print("\n⚠️ WARNING: Normalization factor (sum_rated) is 0. Setting nor
  norm_factor = 1.0

C_positive_norm = C_positive[1:] / norm_factor
#C_negative_norm = -C_negative[1:] / norm_factor
H_norm = H / norm_factor
UCL_norm = UCL / norm_factor
# -------------------------
```

```
200
201 # ========================================================================
202 #  Appliance Sum Time Series (Theft Simulation)
203 # ========================================================================
204 print("\nGenerating Time Series Plot for Theft Simulation...")
205 fig, ax = plt.subplots(figsize=(12, 5), dpi=300)
206
207 # The data stream CUSUM monitors is the theft-affected Meter_Reading_Theft
208 ax.plot(df_test['Time'], df_test['Appliance_Sum_with_theft'],
209         c=CL['RED'], linewidth=1.5, label='Appliance sum with Theft')
210
211 # Also plot the original monitored baseline for comparison
212 ax.plot(df_test['Time'], df_test['Appliance_Sum'],
213         c=CL['CYA'], linewidth=1, linestyle='--',
214         label='Appliance Sum (Baseline)')
215
216 ax.set_title(f"Appliance Sum vs. Appliance Sum with theft of Washing Machi
217 ax.set_xlabel('Time', fontsize=12)
218 ax.set_ylabel('Power (Watts)', fontsize=12)
219 ax.legend(loc='upper left')
220 ax.grid(linestyle='--')
221 plt.tight_layout()
222
223 # Save this time series plot
224 time_series_filename = 'test_data_with_theft_profile.png'
225 plt.savefig(time_series_filename, dpi=300)
226 print(f"Time series plot saved to: {time_series_filename}")
227
228 ax.tick_params(axis='x', labelsize=6)
229 plt.show()
230
231 # ========================================================================
232 #  Plotting (Modern, High Quality & Smaller)
233 # ========================================================================
234 print("\nGenerating modern, compact plot...")
235
236
237 # 1. Reduced figsize to (6, 4) for compactness; kept high dpi (300)
238 fig, ax = plt.subplots(figsize=(6, 4), dpi=300)
239
240
241 # The time axis N now represents the count of *test* samples
242 # Use lighter line weights (lw) for a cleaner, modern look
243 ax.hlines([UCL_norm], N[0] - 10, N[-1] + 10, colors=CL['RED'], linestyles=
244 ax.hlines([0], N[0] - 10, N[-1] + 10, colors='black', linestyles='--', lab
245 ax.plot(N, C_positive_norm, c=CL['BLU'], label='$C^+$', lw=1.8, alpha=0.9)
246
247 # Note: The 'fivethirtyeight' style handles grid and background automatica
248 ax.set_xlim([0, N[-1]])
249
250 # Adjusted font sizes to fit the smaller plot while remaining readable
```

```
ax.set_xlabel('Test Samples', fontsize=12)
ax.set_ylabel('CUSUM values ', fontsize=12)

# Added a clear title
ax.set_title('CUSUM Detection with theft of Washing machine and Kittle', f

# Increased axis tick label size for readability
ax.tick_params(axis='both', which='major', labelsize=10)

ax.legend(loc='upper left', fontsize=8)

ax.grid(True, which='major', linestyle='--', linewidth=0.5, color='gray',

# This ensures the grid stays BEHIND your blue line
ax.set_axisbelow(True)

# Removed ax.grid() because 'fivethirtyeight' adds a light grid by default
plt.tight_layout()

# 4. Save the figure to a high-quality file before showing
output_filename = 'cusum_plot_modern_compact.png'
plt.savefig(output_filename, dpi=300, bbox_inches='tight')
print(f"High-quality plot saved to: {output_filename}")

plt.show() # Still show the plot on screen
# Reset style after plotting
plt.style.use('default')

# ================================================================================
# SAVE RESULTS (JSON FOR DASHBOARD)
# ================================================================================
print("\n=== Saving Dashboard Data ===")

# --- 1. APPLY NORMALIZATION FOR JSON OUTPUT ---
# ⚠️ CORRECTED: Using norm_factor (sum_rated) for consistency with the pl
# The previous logic used H_safe, which was different.
C_positive_norm_json = C_positive[1:] / norm_factor # Skip the initial 0 v

# --- 2. Create DataFrame in requested format ---
dashboard_data_df = pd.DataFrame({
  # 'sample' starts from 1, matching the required JSON format (N is 0-inde
  'sample': N + 1,
  # 'value' is the normalized CUSUM trace
  'value': C_positive_norm_json
})

# Convert DataFrame to JSON (list of objects format)
try:
  dashboard_json = dashboard_data_df.to_json(orient='records')

  # Save the JSON string to a file
```

```
  file_name = "cusum_dashboard_data_test_subset.json" # Changed filename t
  with open(file_name, "w") as f:
    f.write(dashboard_json)

  print(f"CUSUM data saved to: {file_name}")

except Exception as e:
  print(f"Error saving CUSUM dashboard data: {e}")
  print("Ensure all calculated values are valid (e.g., no NaNs or infs).")

print("\n=== Process Complete ===")
```

```
Using 1048575 samples for initial processing (100% subset of original data)
--- Individual Appliance Rated Power (Max Observed) ---
Appliance2: 2677.00
Appliance3: 2385.00
Appliance8: 2933.00
```

Theft of all the three Appliances

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

# Set global font preference
plt.rcParams.update({
    "font.family": "sans-serif"
})

# Color Palette
CL = {
    'BLU': '#0077BB',
    'CYA': '#33BBEE',
    'MAG': '#EE3377',
    'RED': '#CC3311',
}

""" Import & Initial Processing """
# Read the data, ensuring the 'Time' column is used for date parsing
file_path = 'House_2.csv'
if not os.path.exists(file_path):
    print(f"Error: {file_path} not found. Please ensure the file is in the

# Read the data
df = pd.read_csv(file_path)

# Convert the 'Time' column to datetime objects
df['Time'] = pd.to_datetime(df['Time'])

# Use 100% of data for processing
df = df.iloc[:int(len(df) * 1)]
print(f"Using {len(df)} samples for initial processing (100% subset of ori

appliance_numbers = [2, 3, 8]

# Finding rating of all appliances
appliance_cols = [f'Appliance{i}' for i in appliance_numbers]

# Calculate the maximum rated power
appliance_max_ratings = df[appliance_cols].max()

print("--- Individual Appliance Rated Power (Max Observed) ---")
for col, rating in appliance_max_ratings.items():
    print(f"{col}: {rating:.2f}")

```

```
sum_rated = appliance_max_ratings.sum()
print(f"\nTotal Sum of Rated Power (sum_rated): {sum_rated:.2f}")

# Calculating appliance sum
print(f"Creating 'Appliance_Sum' from: {appliance_cols}")
df['Appliance_Sum'] = df[appliance_cols].sum(axis=1)

# --- DOWNSAMPLING ---
sampling_rate = 75 # read every 10 minutes
df = df.iloc[::sampling_rate, :].reset_index(drop=True)
print(f"Downsampling applied: Using 1 sample every {sampling_rate} samples

# --- Weekend Exclusion Logic ---
days_to_keep = [0, 1, 2, 3] # Monday, Tuesday, Wednesday, Thursday
df_weekday = df[df['Time'].dt.dayofweek.isin(days_to_keep)].copy()
print(f"Filtered to include only selected weekdays. Remaining samples: {le

# ============================================================================
#  80/20 Data Split
# ============================================================================
split_point = int(len(df_weekday) * 0.8)

# 80% for historical data
df_historical = df_weekday.iloc[:split_point].copy()
# 20% for test data
df_test = df_weekday.iloc[split_point:].copy()

print(f"\n--- Data Split ---")
print(f"Historical Data: {len(df_historical)} samples")
print(f"Test Data: {len(df_test)} samples")
print(f"------------------------------------\n")

# ============================================================================
#  CUSUM Parameter Calculation (Using Historical Data)
# ============================================================================
sigma_0 = df_historical['Appliance_Sum'].std()
mu_0 = df_historical['Appliance_Sum'].mean()
print(f"mu_0 (Historical): {mu_0:.2f}W, sigma_0 (Historical): {sigma_0:.2f

if sigma_0 == 0:
    print("Warning: sigma_0 is zero. Setting default k and H.")
    b = 8
    k = b
    H = 5
else:
    if sigma_0 > 3 * mu_0:
        b = 1.2
    elif sigma_0 > 2 * mu_0:
        b = 1.2
    elif sigma_0 > mu_0:
        b = 4.5
```

```
    else:
        b = 8
    k = b * sigma_0
    H = 5 * sigma_0

# Define the shift (mu_1) to detect.
mu_1 = mu_0 * 1.10
K = np.fabs(mu_1 - mu_0) / 2

if K == 0 or sigma_0 == 0:
    UCL = float('inf')
else:
    UCL = mu_0 + sigma_0 * (k/K)

print(f"CUSUM Parameters (Historical): K={K:.2f}W, H={H:.2f}W")
print(f"UCL (mean + K*sigma): {UCL:.2f}W")

# ================================================================================
#  Add Theft Data (Simulation: All 3 Appliances)
# ================================================================================
noise_vector = np.random.normal(loc=0, scale=sigma_0 * 0.2, size=len(df_te

# Define the columns that represent the stolen load
# Doubling ALL 3 appliances to simulate major theft
all_columns_for_theft_sum = [
  'Appliance2', 'Appliance3', 'Appliance8',
  'Appliance2', 'Appliance3', 'Appliance8'
]

df_test['Appliance_Sum_with_theft'] = (df_test[all_columns_for_theft_sum].

sigma_x = df_test['Appliance_Sum_with_theft'].std()
mu_x = df_test['Appliance_Sum_with_theft'].mean()
print(f'mu_x (Theft): {mu_x}, sigma_x (Theft): {sigma_x}')

# ================================================================================
#  Process (CUSUM on Test Data WITH THEFT)
# ================================================================================
sample_data = df_test["Appliance_Sum_with_theft"].values
N = np.arange(len(sample_data))

C_positive = [0]

for i in range(N.shape[0]):
    # CUSUM calculation on the *test* raw data
    C_positive_new = np.max([0, sample_data[i] - (mu_0 + K) + C_positive[i
    C_positive.append(C_positive_new)

C_positive = np.asarray(C_positive)

# --- NORMALIZATION STEP ---
```

```
norm_factor = UCL

if norm_factor == 0:
    print("\n⚠️ WARNING: Normalization factor is 0. Setting to 1.0.")
    norm_factor = 1.0

C_positive_norm = C_positive[1:] / norm_factor
H_norm = H / norm_factor
UCL_norm = UCL / norm_factor

# ==========================================================================
#  Time Series Plot (Theft Simulation)
# ==========================================================================
print("\nGenerating Time Series Plot for Theft Simulation...")
fig, ax = plt.subplots(figsize=(12, 5), dpi=300)

ax.plot(df_test['Time'], df_test['Appliance_Sum_with_theft'],
        c=CL['RED'],  linewidth=1.5, label='Appliance sum with Theft')

ax.plot(df_test['Time'], df_test['Appliance_Sum'],
        c=CL['CYA'], linewidth=1, linestyle='--',
        label='Appliance Sum (Baseline)')

ax.set_title(f"Appliance Sum vs. Theft (3 Appliances)", fontsize=16)
ax.set_xlabel('Time', fontsize=12)
ax.set_ylabel('Power (Watts)', fontsize=12)
ax.legend(loc='upper left')
ax.grid(linestyle='--')
plt.tight_layout()

time_series_filename = 'test_data_with_theft_profile.png'
plt.savefig(time_series_filename, dpi=300)
print(f"Time series plot saved to: {time_series_filename}")

ax.tick_params(axis='x', labelsize=8)
plt.show()

# ==========================================================================
#  Plotting (Modern, High Quality & Smaller) - THEFT DETECTED
# ==========================================================================
print("\nGenerating modern, compact plot...")

# 1. Reduced figsize to (6, 4) for compactness; kept high dpi (300)
fig, ax = plt.subplots(figsize=(6, 4), dpi=300)

# The time axis N represents the count of *test* samples
ax.hlines([UCL_norm], N[0] - 10, N[-1] + 10, colors=CL['RED'], linestyles=
ax.hlines([0], N[0] - 10, N[-1] + 10, colors='black', linestyles='--', lab
ax.plot(N, C_positive_norm, c=CL['BLU'], label='$C^+$', lw=1.8, alpha=0.9)

ax.set_xlim([0, N[-1]])
```

```
200
201 # Adjusted font sizes to fit the smaller plot while remaining readable
202 ax.set_xlabel('Test Samples', fontsize=12)
203 ax.set_ylabel('CUSUM values', fontsize=12)
204
205 # Added a clear title
206 ax.set_title('CUSUM Detection (Theft equal to our house)', fontsize=14, pa
207
208 # Increased axis tick label size for readability
209 ax.tick_params(axis='both', which='major', labelsize=10)
210
211 ax.legend(loc='upper left', fontsize=8)
212
213 ax.grid(True, which='major', linestyle='--', linewidth=0.5, color='gray',
214
215 # This ensures the grid stays BEHIND your blue line
216 ax.set_axisbelow(True)
217
218 plt.tight_layout()
219
220 # Save the figure
221 output_filename = 'cusum_plot_modern_theft_all3.png'
222 plt.savefig(output_filename, dpi=300, bbox_inches='tight')
223 print(f"High-quality plot saved to: {output_filename}")
224
225 plt.show()
226 plt.style.use('default')
227
228 # ================================================================================
229 # SAVE RESULTS (JSON FOR DASHBOARD)
230 # ================================================================================
231 print("\n=== Saving Dashboard Data ===")
232
233 C_positive_norm_json = C_positive[1:] / norm_factor
234
235 dashboard_data_df = pd.DataFrame({
236     'sampleNumber': N + 1,
237     'cusumScore': C_positive_norm_json
238 })
239
240 try:
241     dashboard_json = dashboard_data_df.to_json(orient='records')
242     file_name = "cusum_dashboard_data_theft_all3.json"
243     with open(file_name, "w") as f:
244         f.write(dashboard_json)
245     print(f"CUSUM data saved to: {file_name}")
246
247 except Exception as e:
248     print(f"Error saving CUSUM dashboard data: {e}")
249
250 print("\n=== Process Complete ===")
```

```
Using 1048575 samples for initial processing (100% subset of original data)
--- Individual Appliance Rated Power (Max Observed) ---
Appliance2: 2497.00
Appliance3: 2385.00
Appliance8: 2933.00
```

With cross validation for choosing k values

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

plt.rcParams.update({
    "font.family": "sans-serif"
})


CL = {
    'BLU': '#0077BB', # blue
    'CYA': '#33BBEE', # cyan
    'MAG': '#EE3377', # magenta
    'RED': '#CC3311', # red
}

# --- Utility Functions for Cross-Validation ---

def run_cusum(data, data_column, mu_0, K):
    """Runs the CUSUM calculation for C_positive."""
    sample_data = data[data_column].values
    C_positive = [0]

    for x_i in sample_data:
        # CUSUM calculation for upward shift (theft)
        C_positive_new = np.max([0, x_i - (mu_0 + K) + C_positive[-1]])
        C_positive.append(C_positive_new)

    # Return C_positive (excluding the initial zero)
    return np.asarray(C_positive[1:])

def evaluate_performance(CUSUM_values, UCL_threshold):
    """
    Runs the detection and calculates False Alarm Rate (FAR) using UCL as
    """
    # Detection: An alarm is triggered if CUSUM exceeds UCL
    alarms = CUSUM_values > UCL_threshold

    alarms_count = alarms.sum()
    total_samples = len(CUSUM_values)

    # Calculate False Alarm Rate (FAR) - Alarms / Total Samples (Minimizir
    FAR = alarms_count / total_samples if total_samples > 0 else 0

    return {
```

```
        'alarms_count': alarms_count,
        'FAR': FAR
    }
# --- End of Utility Functions ---


""" Import & Initial Processing """
file_path = 'House_2.csv'
if not os.path.exists(file_path):
    print(f"Error: {file_path} not found. Please ensure the file is in the
    exit()

df = pd.read_csv(file_path)
df['Time'] = pd.to_datetime(df['Time'])
df = df.iloc[:int(len(df) * 1)]
print(f"Using {len(df)} samples for initial processing (100% subset of ori

appliance_numbers = [ 2, 3, 8]
appliance_cols = [f'Appliance{i}' for i in appliance_numbers]

# Finding rating and sum_rated
appliance_max_ratings = df[appliance_cols].max()
sum_rated = appliance_max_ratings.sum()
print(f"Total Sum of Rated Power (sum_rated): {sum_rated:.2f}")

# Appliance Sum
print(f"Creating 'Appliance_Sum' from: {appliance_cols}")
df['Appliance_Sum']=df[appliance_cols].sum(axis=1)

# Downsampling
sampling_rate = 75
df = df.iloc[::sampling_rate, :].reset_index(drop=True)
print(f"Downsampling applied: Using 1 sample every {sampling_rate} samples

# Weekend Exclusion
days_to_keep = [0, 1, 2, 3] # Monday to Thursday
df_weekday = df[df['Time'].dt.dayofweek.isin(days_to_keep)].copy().reset_i
print(f"Filtered to include only selected weekdays. Remaining samples: {le

# ================================================================================
# 🎯 Cross-Validation Setup (Walk-Forward) 🎯
# ================================================================================
#
# 1. Define the range of factors (b) to test, where k = b * sigma_0
# We test a sensible range of factors (b) for k.
b_factors = np.arange(0.5, 9.1, 0.5)

N_FOLDS = 5 # Number of folds for the total dataset
FOLD_SIZE = int(len(df_weekday) / N_FOLDS)
RESULTS = []

```

```
print(f"\n--- Cross-Validation Setup: Walk-Forward Validation ({N_FOLDS} f
print(f"Testing factor 'b' for k = b * sigma_0: {b_factors}")


for i in range(1, N_FOLDS):
    # Training Data
    train_end_index = FOLD_SIZE * i
    df_historical_fold = df_weekday.iloc[:train_end_index].copy()

    # Test Data
    test_start_index = train_end_index
    test_end_index = train_end_index + FOLD_SIZE
    df_test_fold = df_weekday.iloc[test_start_index:test_end_index].copy()

    if len(df_test_fold) < 1:
        continue

    print(f"\nFold {i}: Train size={len(df_historical_fold)}, Test size={l

    # 2. Calculate CUSUM Parameters (mu_0, sigma_0, K) for the training da
    sigma_0_fold = df_historical_fold['Appliance_Sum'].std()
    mu_0_fold = df_historical_fold['Appliance_Sum'].mean()

    # Define the shift (mu_1) to detect (10% shift as in your original coc
    mu_1_fold = mu_0_fold * 1.10
    K_fold = np.fabs(mu_1_fold - mu_0_fold) / 2 # K is the slack value

    if sigma_0_fold == 0 or K_fold == 0:
        print("Skipping fold due to zero sigma or K.")
        continue

    # 3. Run CUSUM on the Test Data
    C_positive_fold = run_cusum(df_test_fold, 'Appliance_Sum', mu_0_fold,

    # 4. Evaluate Performance for each b_factor
    for b_factor in b_factors:
        # Calculate k
        k_fold = b_factor * sigma_0_fold

        # Calculate UCL using the custom formula: UCL = mu_0 + sigma_0 * (
        UCL_threshold = mu_0_fold + sigma_0_fold * (k_fold / K_fold)

        # Evaluate performance using UCL as the alarm threshold
        metrics = evaluate_performance(C_positive_fold, UCL_threshold)

        RESULTS.append({
            'Fold': i,
            'b_factor': b_factor,
            'UCL_threshold': UCL_threshold,
            'FAR': metrics['FAR']
        })
```

```
149
150 # 5. Analyze Results and Select Optimal Factor 'b'
151 results_df = pd.DataFrame(RESULTS)
152 summary_results = results_df.groupby('b_factor').agg(
153     Avg_FAR=('FAR', 'mean'),
154     Std_FAR=('FAR', 'std'),
155 ).reset_index()
156
157 print("\n--- Cross-Validation Summary (Optimizing for Min FAR) ---")
158 print(summary_results.to_string(index=False))
159
160 # Select the b_factor that minimizes the Average False Alarm Rate (FAR)
161 optimal_factor_row = summary_results.loc[summary_results['Avg_FAR'].idxmir
162 OPTIMAL_B_FACTOR = optimal_factor_row['b_factor']
163
164 print(f"\n✅ Optimal factor 'b' (Minimizing Avg FAR): {OPTIMAL_B_FACTOR:.
165
166 # ========================================================================
167 ## 🚀 Final CUSUM Run with Optimal Parameter (on the 20% test data) 🚀
168 # ========================================================================
169
170 # Recalculate the 80/20 split based on df_weekday
171 split_point = int(len(df_weekday) * 0.8)
172 df_historical_final = df_weekday.iloc[:split_point].copy()
173 df_test_final = df_weekday.iloc[split_point:].copy()
174
175 # Use the 80% split for final parameters
176 sigma_0_final = df_historical_final['Appliance_Sum'].std()
177 mu_0_final = df_historical_final['Appliance_Sum'].mean()
178 mu_1_final = mu_0_final * 1.10
179 K_final = np.fabs(mu_1_final - mu_0_final) / 2
180
181 # Use the OPTIMAL_B_FACTOR found via Cross-Validation
182 k_final = OPTIMAL_B_FACTOR * sigma_0_final
183
184 # Calculate the final UCL (which serves as the alarm threshold)
185 UCL_final = mu_0_final + sigma_0_final * (k_final / K_final)
186
187 # --- Theft Simulation ---
188 # Applying theft load to the TEST data
189 noise_vector = np.random.normal(loc=0, scale=sigma_0_final * 0.2, size=ler
190 all_columns_for_theft_sum = [
191     'Appliance2', 'Appliance3', 'Appliance8',
192     'Appliance2','Appliance3','Appliance8'
193 ]
194 df_test_final['Appliance_Sum_with_theft'] = (df_test_final[all_columns_for
195 # ------------------------
196
197 print(f"\nFinal CUSUM Run Parameters (80% Historical Data):")
198 print(f"mu_0: {mu_0_final:.2f}W, sigma_0: {sigma_0_final:.2f}W")
199 print(f"K (Slack): {K_final:.2f}W, Optimal k: {k_final:.2f}W (from {OPTIMA
```

```
print(f"Final UCL Threshold: {UCL_final:.2f}W")


# 🎯 Rerun CUSUM on the *theft-affected* test data
C_positive = run_cusum(df_test_final, 'Appliance_Sum_with_theft', mu_0_fir
N = np.arange(len(C_positive))


# --- FINAL NORMALIZATION AND PLOTTING ---
# Normalizing by the final UCL value
norm_factor = UCL_final
if norm_factor == 0: norm_factor = 1.0

C_positive_norm = C_positive / norm_factor
UCL_norm = 1.0 # UCL is 1.0 after normalizing by itself
# ----------------------------------------

print("\nGenerating final plot with optimized UCL threshold...")
# Apply a modern style for aesthetics
plt.style.use('fivethirtyeight')

fig, ax = plt.subplots(figsize=(6, 4), dpi=300)

# The time axis N now represents the count of *test* samples
ax.hlines([UCL_norm], N[0] - 10, N[-1] + 10, colors=CL['RED'], linestyles=
ax.hlines([0], N[0] - 10, N[-1] + 10, colors='black', linestyles='--', lab
ax.plot(N, C_positive_norm, c=CL['BLU'], label='$C^+$', lw=1.8, alpha=0.9)

ax.set_xlim([0, N[-1]])
ax.set_xlabel('Test Samples', fontsize=12)
ax.set_ylabel('CUSUM values (Normalized by UCL)', fontsize=12)
ax.set_title(f'CUSUM Detection - Optimal UCL Factor (b={OPTIMAL_B_FACTOR:.

ax.tick_params(axis='both', which='major', labelsize=10)
ax.legend(loc='upper right', fontsize=8)
plt.tight_layout()

output_filename = 'cusum_plot_optimized_UCL.png'
plt.savefig(output_filename, dpi=300, bbox_inches='tight')
print(f"High-quality plot saved to: {output_filename}")

plt.show()

# Reset style after plotting
plt.style.use('default')

# --- Time Series Plot ---
print("\nGenerating Time Series Plot for Theft Simulation...")
fig, ax = plt.subplots(figsize=(12, 5), dpi=300)

ax.plot(df_test_final['Time'], df_test_final['Appliance_Sum_with_theft'],
```

```
        c=CL['RED'], linewidth=1.5, label='Appliance sum with Theft')

ax.plot(df_test_final['Time'], df_test_final['Appliance_Sum'],
        c=CL['CYA'], linewidth=1, linestyle='--',
        label='Appliance Sum (Baseline)')

ax.set_title(f"Appliance Sum vs. Appliance Sum with Theft Simulation", for
ax.set_xlabel('Time', fontsize=12)
ax.set_ylabel('Power (Watts)', fontsize=12)
ax.legend(loc='upper left')
ax.grid(linestyle='--')
plt.tight_layout()

time_series_filename = 'test_data_with_theft_profile.png'
plt.savefig(time_series_filename, dpi=300)
print(f"Time series plot saved to: {time_series_filename}")

plt.show()
```

```
<>:199: SyntaxWarning: invalid escape sequence '\s'
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

plt.rcParams.update({
    "font.family": "sans-serif"
})


CL = {
    'BLU': '#0077BB', # blue
    'CYA': '#33BBEE', # cyan
    'MAG': '#EE3377', # magenta
    'RED': '#CC3311', # red
}

""" Import & Initial Processing """
# Read the data, ensuring the 'Time' column is used for date parsing
file_path = 'House_2.csv'
if not os.path.exists(file_path):
    # This will prevent the code from running if the file is missing
    print(f"Error: {file_path} not found. Please ensure the file is in the
    # raise FileNotFoundError(f"Error: {file_path} not found.")

# Read the data. Assuming 'Time' is the first column (index 0).
df = pd.read_csv(file_path)

# 🚀 Convert the 'Time' column to datetime objects
df['Time'] = pd.to_datetime(df['Time'])

# 🎯 Use 100% of data for processing
df = df.iloc[:int(len(df) * 1)]
# ⚠️ CORRECTED PRINT STATEMENT (was 8%, now 100%)
print(f"Using {len(df)} samples for initial processing (100% subset of ori

appliance_numbers = [2, 3, 8]


# Finding rating of all appliances
appliance_cols = [f'Appliance{i}' for i in appliance_numbers]

# 2. Calculate the maximum rated power for ALL selected columns in one go.
appliance_max_ratings = df[appliance_cols].max()

# 3. Print the individual rated values using iteration
print("--- Individual Appliance Rated Power (Max Observed) ---")
for col, rating in appliance_max_ratings.items():
    print(f"{col}: {rating:.2f}")

```

```
# 4. Calculate the total sum directly from the Series (sum_rated)
sum_rated = appliance_max_ratings.sum()
print(f"\nTotal Sum of Rated Power (sum_rated): {sum_rated:.2f}")

#5. Calculating appliance sum for the three appliance to test the cusum or
print(f"Creating 'Appliance_Sum' from: {appliance_cols}")
df['Appliance_Sum']=df[appliance_cols].sum(axis=1)



#""" Data Cleaning """

#outlier_threshold = 14000 # Threshold for extreme outliers (in Watts)
#print(f"Cleaning outliers above {outlier_threshold}W using interpolation.
#df['Aggregate'] = np.where(df['Aggregate'] > outlier_threshold, np.nan, d
#df['Aggregate'] = df['Aggregate'].interpolate(method='linear')
#df = df.dropna(subset=['Aggregate']).reset_index(drop=True)

# --- DOWNSAMPLING APPLIED HERE ---
sampling_rate = 35 #read every 10 minutes
df = df.iloc[::sampling_rate, :].reset_index(drop=True)
print(f"Downsampling applied: Using 1 sample every {sampling_rate} samples
# --- END OF DOWNSAMPLING ---


## 🚀 Weekend Exclusion Logic 🚀 ##
# Note: pandas dayofweek is 0=Monday, 6=Sunday.
days_to_keep = [0, 1, 2, 3] # Monday, Tuesday, Wednesday, Thursday

# Filter the DataFrame using the 'Time' column
df_weekday = df[df['Time'].dt.dayofweek.isin(days_to_keep)].copy()
print(f"Filtered to include only selected weekdays. Remaining samples: {le

# ================================================================================
#  80/20 Data Split for Historical (Training) and Test Data
# (Applied to the existing 'df_weekday' subset)
# ================================================================================

# Calculate split point (80% for historical, 20% for test)
split_point = int(len(df_weekday) * 0.6)

# 80% for historical data (for calculating mu_0 and sigma_0)
df_historical = df_weekday.iloc[:split_point].copy()
# 20% for test data (for CUSUM monitoring)
df_test = df_weekday.iloc[split_point:].copy()

print(f"\n--- Data Split (from the 100% subset) ---")
print(f"Historical Data (80% of subset): {len(df_historical)} samples")
print(f"Test Data (20% of subset): {len(df_test)} samples")
print(f"----------------------------------------\n")

```

```
102
103
104
105 # ========================================================================
106 # 🎯 CUSUM Parameter Calculation (Using Historical Data)
107 # ========================================================================
108 # Parameters must now be calculated based on the *historical* data (in Wat
109 sigma_0 = df_historical['Appliance_Sum'].std()
110 mu_0 = df_historical['Appliance_Sum'].mean()
111 print(f"mu_0 (Historical): {mu_0:.2f}W, sigma_0 (Historical): {sigma_0:.2f
112
113 # ⚠️ Handle the case where sigma_0 is zero to prevent division by zero in
114 if sigma_0 == 0:
115     print("Warning: sigma_0 is zero. Setting default k and H to prevent er
116     b = 8
117     k = b
118     H = 5
119 else:
120     if sigma_0 > 3 * mu_0:
121         b = 0.7
122     elif sigma_0 > 2 * mu_0:
123         b = 1.2 # User-defined value
124     elif sigma_0 > mu_0:
125         b = 4.5
126     else:
127         b = 8
128     k = b * sigma_0
129     H = 5 * sigma_0 # H is the decision interval (control limit) (in Watts
130
131 # Define the shift (mu_1) to detect. Example: a 110% shift of the mean.
132 mu_1 = mu_0 * 1.05
133 K = np.fabs(mu_1 - mu_0) / 2 # K is the slack value (in Watts)
134
135
136 # Calculation of UCL (LCL calculation is removed)
137 # Ensure K is not zero before division
138 if K == 0 or sigma_0 == 0:
139     UCL = float('inf') # Set to a very high value if K or sigma_0 is zero
140 else:
141     UCL = mu_0 + sigma_0 * (k/K)
142
143 print(f"CUSUM Parameters (Historical): K={K:.2f}W, H={H:.2f}W")
144 print(f"UCL (mean + K*sigma): {UCL:.2f}W")
145
146
147 #Adding the theft data with a noise
148
149 noise_vector = np.random.normal(loc=0, scale=sigma_0 * 0.1, size=len(df_te
150
151 # Assuming appliance_cols = ['Appliance2', 'Appliance3', 'Appliance8']
152
```

```
# Define the columns that represent the stolen load
theft_columns = ['Appliance2', 'Appliance3','Appliance8']
all_columns_to_sum = appliance_cols + theft_columns
all_columns_to_sum = list(set(all_columns_to_sum))

# --- FIX 2: Perform the operation ONLY on df_test ---
# The result should be a new column on df_test, with length 814.
# We are creating a new column on df_test called 'Appliance_Sum_with_theft
df_test['Appliance_Sum_with_theft'] = (df_test[all_columns_to_sum].sum(axi

sigma_x=df_test['Appliance_Sum_with_theft'].std()
print(f'Standard deviation of Appliance_Sum_with_theft: {sigma_x}')

mu_x=df_test['Appliance_Sum_with_theft'].mean()
print(f'mu_x: {mu_x}')
# ============================================================================
# 🎯 Process (CUSUM on Test Data)
# ============================================================================
# Use the test data for CUSUM calculation
sample_data = df_test["Appliance_Sum_with_theft"].values
N = np.arange(len(sample_data)) # N is now the index of the TEST data

C_positive = [0]
C_negative = [0]

for i in range(N.shape[0]):
    # CUSUM calculation on the *test* raw data (Watts)
    C_positive_new = np.max([0, sample_data[i] - (mu_0 + K) + C_positive[i
    C_positive.append(C_positive_new)
    #C_negative_new = np.max([0, (mu_0 - K) - sample_data[i] + C_negative[
    #C_negative.append(C_negative_new)

C_positive = np.asarray(C_positive)
#C_negative = np.asarray(C_negative)

# --- NORMALIZATION STEP ---
# Use the calculated sum_rated as the normalization factor.
norm_factor = sum_rated

# Handle potential zero division
if norm_factor == 0:
    print("\n⚠️ WARNING: Normalization factor (sum_rated) is 0. Setting n
    norm_factor = 1.0

C_positive_norm = C_positive[1:] / norm_factor
#C_negative_norm = -C_negative[1:] / norm_factor
H_norm = H / norm_factor
UCL_norm = UCL / norm_factor
# --------------------------

```

```
# ==============================================================================
# 🖼️ Plotting (High Quality & Clarity)
# ==============================================================================
print("\nGenerating high-quality plot...")

# 1. Increased figsize (10, 6) and dpi (300)
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)

# The time axis N now represents the count of *test* samples
ax.hlines([UCL_norm], N[0] - 10, N[-1] + 10, colors=CL['RED'], linestyles=
ax.hlines([0], N[0] - 10, N[-1] + 10, colors=CL['MAG'], linestyles='dashed
ax.plot(N, C_positive_norm, '.-', c=CL['BLU'], label='$C^+$', lw=3, ms=8,
# ax.plot(N, C_negative_norm, '.-', c=CL['CYA'], label='$C^-$', lw=3, ms=8

ax.set_xlim([0, N[-1]])
# Note the label change to reflect the test data
ax.set_xlabel('Test Samples', fontsize=20)
ax.set_ylabel('CUSUM values (Normalized)', fontsize=20)

# 2. Added a clear title
ax.set_title('CUSUM Control Chart (using 3 Appliances)', fontsize=22)

# 3. Increased axis tick label size for readability
ax.tick_params(axis='both', which='major', labelsize=14)

ax.legend(loc='lower right', fontsize=12)
ax.grid(linestyle='--')
plt.tight_layout()

# 4. Save the figure to a high-quality file before showing
output_filename = 'cusum_plot_high_quality_appliance_sum.png'
plt.savefig(output_filename, dpi=300, bbox_inches='tight')
print(f"High-quality plot saved to: {output_filename}")

plt.show() # Still show the plot on screen


# ==============================================================================
# SAVE RESULTS (JSON FOR DASHBOARD)
# ==============================================================================
print("\n=== Saving Dashboard Data ===")

# --- 1. APPLY NORMALIZATION FOR JSON OUTPUT ---
# ⚠️ CORRECTED: Using norm_factor (sum_rated) for consistency with the pl
# The previous logic used H_safe, which was different.
C_positive_norm_json = C_positive[1:] / norm_factor # Skip the initial 0 v

# --- 2. Create DataFrame in requested format ---
dashboard_data_df = pd.DataFrame({
    # 'sample' starts from 1, matching the required JSON format (N is 0-ir
    'sample': N + 1,
```

```
    # 'value' is the normalized CUSUM trace
    'value': C_positive_norm_json
})

# Convert DataFrame to JSON (list of objects format)
try:
    dashboard_json = dashboard_data_df.to_json(orient='records')

    # Save the JSON string to a file
    file_name = "cusum_dashboard_data_test_subset.json" # Changed filename
    with open(file_name, "w") as f:
        f.write(dashboard_json)

    print(f"CUSUM data saved to: {file_name}")

except Exception as e:
    print(f"Error saving CUSUM dashboard data: {e}")
    print("Ensure all calculated values are valid (e.g., no NaNs or infs).

print("\n=== Process Complete ===")
```

```
Using 1048575 samples for initial processing (100% subset of original data)
--- Individual Appliance Rated Power (Max Observed) ---
Appliance2: 2571.00
Appliance3: 2385.00
Appliance8: 2933.00

Total Sum of Rated Power (sum_rated): 7889.00
Creating 'Appliance_Sum' from: ['Appliance2', 'Appliance3', 'Appliance8']
Downsampling applied: Using 1 sample every 35 samples. New sample count: 29960
Filtered to include only selected weekdays. Remaining samples: 17094

--- Data Split (from the 100% subset) ---
Historical Data (80% of subset): 10256 samples
Test Data (20% of subset): 6838 samples
----------------------------------------

mu_0 (Historical): 90.62W, sigma_0 (Historical): 445.54W
CUSUM Parameters (Historical): K=4.53W, H=2227.68W
UCL (mean + K*sigma): 30756.38W
Standard deviation of Appliance_Sum_with_theft: 476.3889108212814
mu_x: 102.50764695545685

Generating high-quality plot...
High-quality plot saved to: cusum_plot_high_quality_appliance_sum.png
```

CUSUM Results